



KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY
AN AUTONOMOUS INSTITUTION - ACCREDITED BY NAAC WITH 'A' GRADE
Narayanaguda, Hyderabad.

Database Management Systems

UNIT-II

UNIT-II

Introduction to the Relational Model: Data Definition Language, Integrity constraint over relations, Types of Integrity Constraints Domain Constraint-String, character, Integer, date, Entity Integrity Constraint-, Primary Key, Referential Integrity Constraint-Foreign Key, Other Key Constraint – NULL,NOT NULL,CHECK and etc , querying relational data, logical data base design, introduction to views, destroying/altering tables and views.

Introduction to the Relational Model

Unit-II

INTRODUCTION TO THE RELATIONAL MODEL

What is Relational Model?

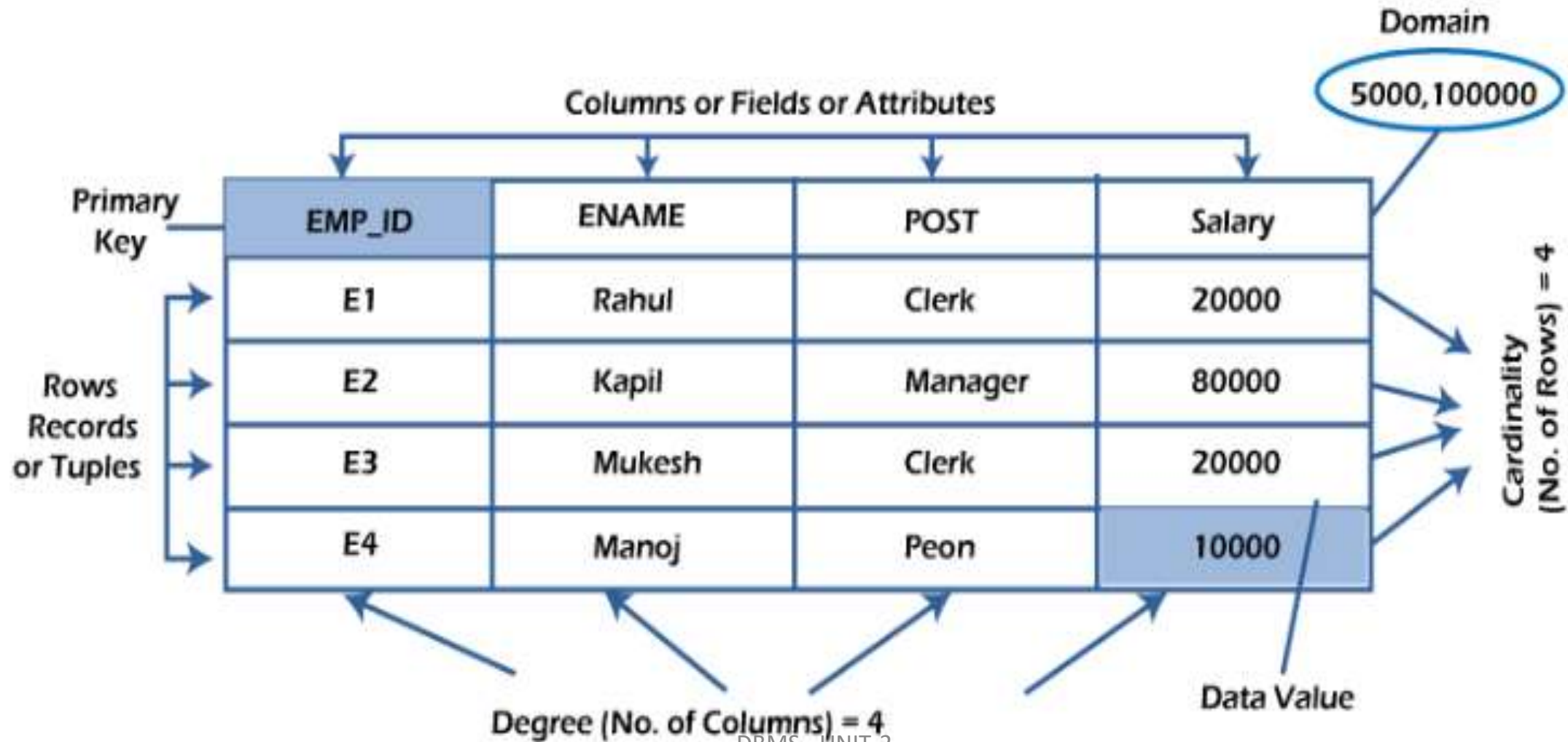
Relational model can be represented as a table with columns and rows. Each row is known as a tuple. Each table of the column has a name or attribute.

Relational Model (RM) represents the database as a collection of relations. A relation is nothing but a table of values. Every row in the table represents a collection of related data values. These rows in the table denote a real-world entity or relationship.

INTRODUCTION TO THE RELATIONAL MODEL

- *The main construct for representing data in the relational model is a relation.*
- *Relational database*: a set of *relations*
- *Relation*: made up of 2 parts:
- *Relation schema and relational instance.*
 - *Instance* : a *table*, with rows and columns.
 - Set of tuples also called as records
 - *#Rows = cardinality, #fields = degree /arity.*
 - A domain is referred by domain name consisting of set of associated values.
 - *Schema* : specifies name of relation, plus name and type of each column.
 - E.G. Students (*sid*: string, *name*: string, *login*: string, *age*: integer, *gpa*: real).
- Can think of a relation as a *set of rows or tuples* (i.e., all rows are distinct).
- All modern database management systems like SQL, MS SQL Server, IBM DB2, ORACLE, My-SQL, and Microsoft Access are based on RDBMS.

various terminologies of RDBMS



Relational Database Management System

- RDBMS stands for **R**elational **D**atabase **M**anagement **S**ystem. RDBMS is the basis for SQL, and for all modern database systems like MS SQL Server, IBM DB2, Oracle, MySQL, and Microsoft Access.
- A Relational database management system (RDBMS) is a database management system (DBMS) that is based on the relational model as introduced by E. F. Codd in 1970.

Example Instance of Students Relation

- An instance of a relation is a set of tuples, also called records, in which each tuple has the same number of fields as the relation schema.
- A relation instance can be thought of as a *table* in which each tuple is a *row*, and all rows have the same number of fields.
- ❖ **Cardinality =3, degree =5, all rows distinct**

sid	name	login	age	gpa
53666	Jones	jones@cs	18	3.4
53688	Smith	smith@eecs	18	3.2
53650	Smith	smith@math	19	3.8

Table also called Relation

Primary Key

Domain
Ex: NOT NULL

CustomerID	CustomerName	Status
1	Google	Active
2	Amazon	Active
3	Apple	Inactive

Tuple OR Row
Total # of rows is **Cardinality**

Column OR Attributes
Total # of column is **Degree**

Relational Model Concepts

- 1. Attribute:** Each column in a Table. Attributes are the properties which define a relation. e.g., Student_Rollno, NAME, etc.
- 2. Tables** – In the Relational model the relations are saved in the table format. It is stored along with its entities. A table has two properties rows and columns. Rows represent records and columns represent attributes.
- 3. Tuple** – It is nothing but a single row of a table, which contains a single record.
- 4. Relation Schema:** A relation schema represents the name of the relation with its attributes.
- 5. Degree (Arity):** The total number of attributes which in the relation is called the degree of the relation.

- 6. Cardinality:** Total number of rows present in the Table.
- 7. Column:** The column represents the set of values for a specific attribute.
- 8. Relation instance** – Relation instance is a finite set of tuples in the RDBMS system. Relation instances never have duplicate tuples.
- 9. Relation key** - Every row has one, two or multiple attributes, which is called relation key.
- 10. Attribute domain** – Every attribute has some pre-defined value and scope which is known as attribute domain.

SQL (Structured Query Language)

SQL (Structured Query Language)

SQL (Structured Query Language) is used to perform operations on the records stored in the database.

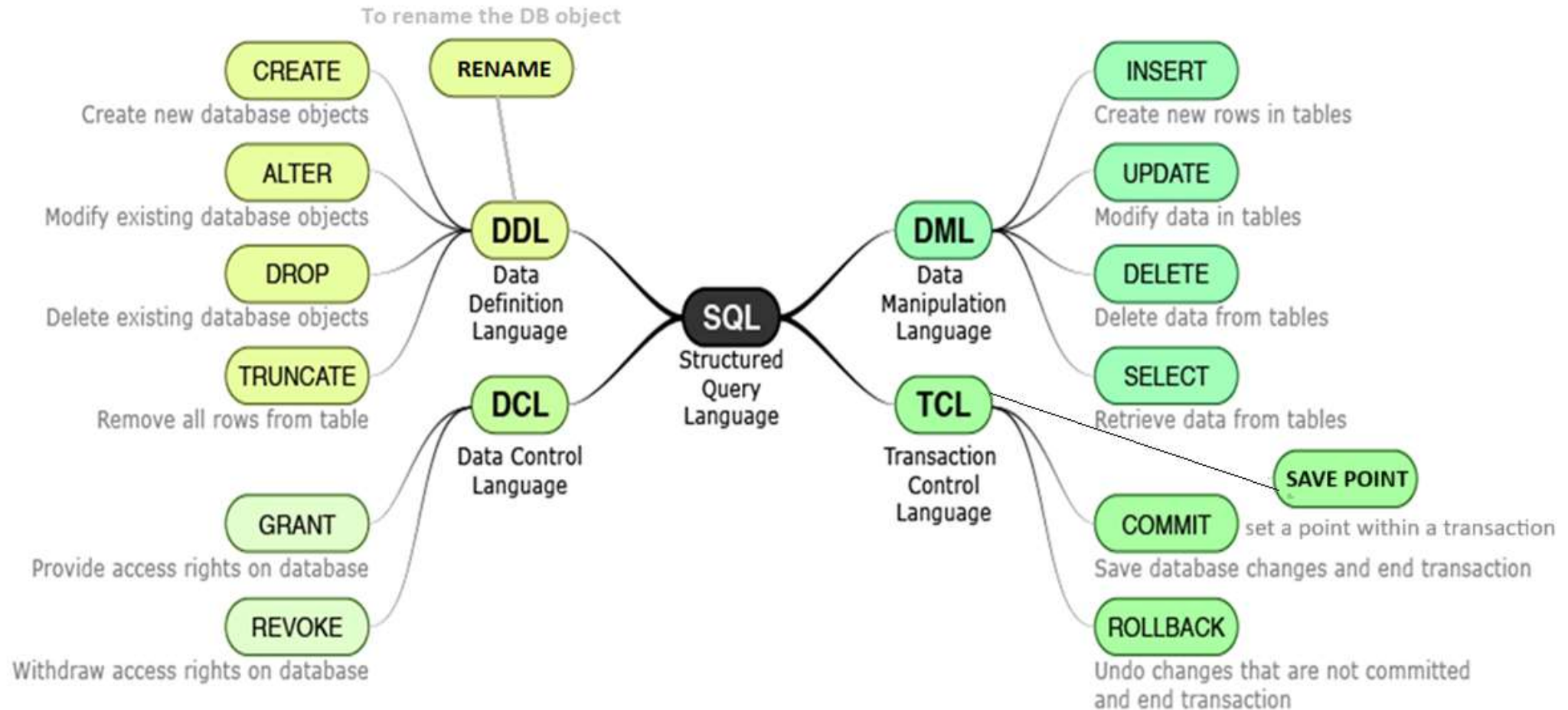
SQL is not a database system, but it is a query language.

SQL is a database computer language designed for the retrieval and management of data in Relational database. **Here, are important reasons for using SQL**

This database language is mainly designed for maintaining the data in relational database management systems. It is a special tool used by data professionals for handling structured data (data which is stored in the form of tables).

This query language became the standard of ANSI in the year of 1986 and ISO in the year of 1987.

SQL commands are divided into four subgroups, DDL, DML, DCL, and TCL



SQL (Structured Query Language)

- To create a column/field, to store, maintain and manipulate data, the sql engine requires a minimum of 3 parameters to be passed by the programmer.

1. Column name

2. Data type

3. Size

MySQL Data Types

- In MySQL there are three main data types: string, numeric, and date and time.
- String Data Types

Data type	Description
CHAR(size)	A FIXED length string (can contain letters, numbers, and special characters). The <i>size</i> parameter specifies the column length in characters - can be from 0 to 255. Default is 1
VARCHAR(size)	A VARIABLE length string (can contain letters, numbers, and special characters). The <i>size</i> parameter specifies the maximum column length in characters - can be from 0 to 65535
BINARY(size)	Equal to CHAR(), but stores binary byte strings. The <i>size</i> parameter specifies the column length in bytes. Default is 1

Numeric Data Types

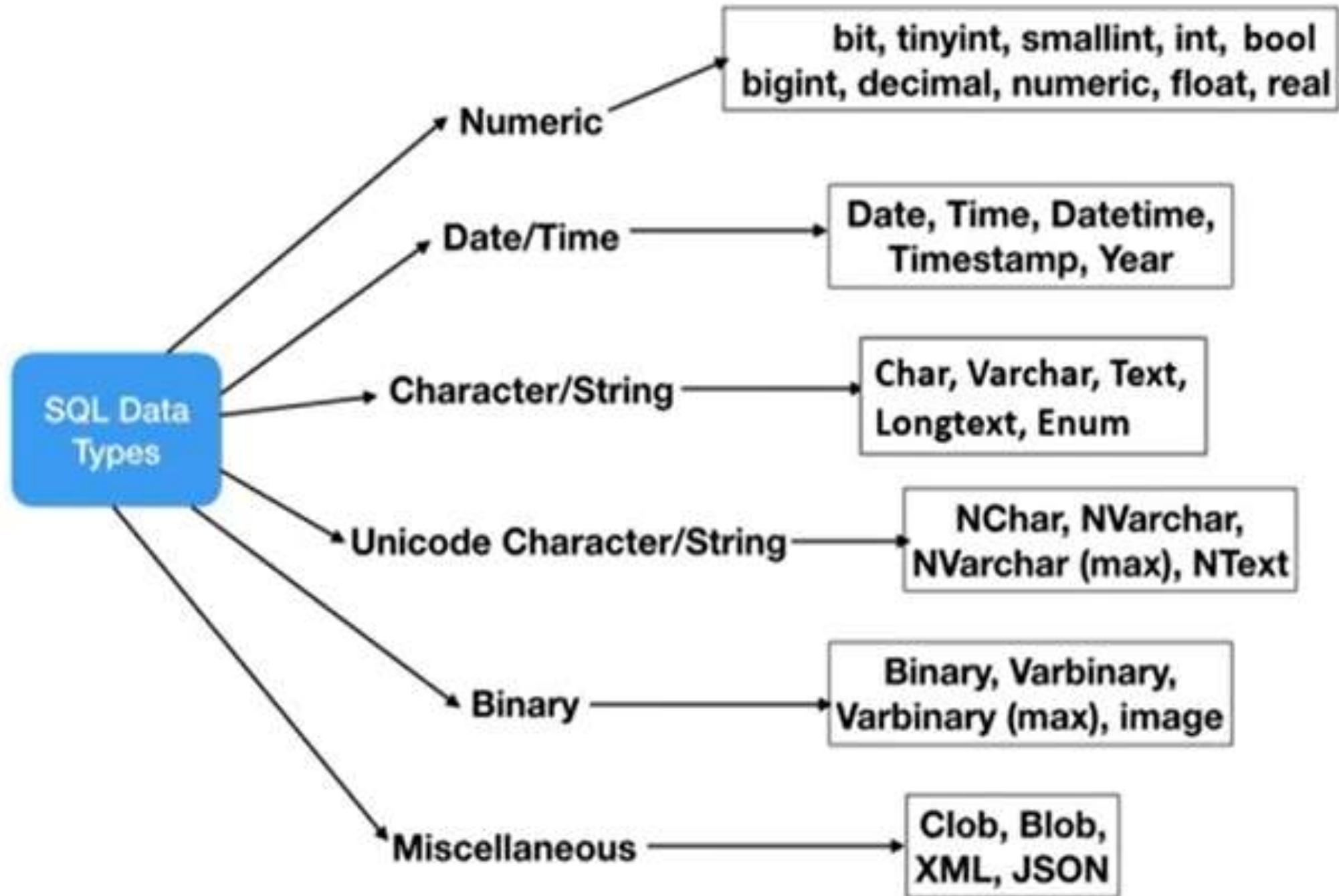
Data type	Description
BIT(<i>size</i>)	A bit-value type. The number of bits per value is specified in <i>size</i> . The <i>size</i> parameter can hold a value from 1 to 64. The default value for <i>size</i> is 1.
TINYINT(<i>size</i>)	A very small integer. Signed range is from -128 to 127. Unsigned range is from 0 to 255. The <i>size</i> parameter specifies the maximum display width (which is 255)
BOOL	Zero is considered as false, nonzero values are considered as true.
BOOLEAN	Equal to BOOL
SMALLINT(<i>size</i>)	A small integer. Signed range is from -32768 to 32767. Unsigned range is from 0 to 65535. The <i>size</i> parameter specifies the maximum display width (which is 255)
MEDIUMINT(<i>size</i>)	A medium integer. Signed range is from -8388608 to 8388607. Unsigned range is from 0 to 16777215. The <i>size</i> parameter specifies the maximum display width (which is 255)
INT(<i>size</i>)	A medium integer. Signed range is from -2147483648 to 2147483647. Unsigned range is from 0 to 4294967295. The <i>size</i> parameter specifies the maximum display width (which is 255)
INTEGER(<i>size</i>)	Equal to INT(<i>size</i>)



BIGINT(<i>size</i>)	A large integer. Signed range is from -9223372036854775808 to 9223372036854775807. Unsigned range is from 0 to 18446744073709551615. The <i>size</i> parameter specifies the maximum display width (which is 255)
FLOAT(<i>size</i> , <i>d</i>)	A floating point number. The total number of digits is specified in <i>size</i> . The number of digits after the decimal point is specified in the <i>d</i> parameter. This syntax is deprecated in MySQL 8.0.17, and it will be removed in future MySQL versions
FLOAT(<i>p</i>)	A floating point number. MySQL uses the <i>p</i> value to determine whether to use FLOAT or DOUBLE for the resulting data type. If <i>p</i> is from 0 to 24, the data type becomes FLOAT(). If <i>p</i> is from 25 to 53, the data type becomes DOUBLE()
DOUBLE(<i>size</i> , <i>d</i>)	A normal-size floating point number. The total number of digits is specified in <i>size</i> . The number of digits after the decimal point is specified in the <i>d</i> parameter
DOUBLE PRECISION(<i>size</i> , <i>d</i>)	
DECIMAL(<i>size</i> , <i>d</i>)	An exact fixed-point number. The total number of digits is specified in <i>size</i> . The number of digits after the decimal point is specified in the <i>d</i> parameter. The maximum number for <i>size</i> is 65. The maximum number for <i>d</i> is 30. The default value for <i>size</i> is 10. The default value for <i>d</i> is 0.
DEC(<i>size</i> , <i>d</i>)	Equal to DECIMAL(<i>size</i> , <i>d</i>)

Date and Time Data Types

Data type	Description
DATE	A date. Format: YYYY-MM-DD. The supported range is from '1000-01-01' to '9999-12-31'
DATETIME(<i>fsp</i>)	A date and time combination. Format: YYYY-MM-DD hh:mm:ss. The supported range is from '1000-01-01 00:00:00' to '9999-12-31 23:59:59'. Adding DEFAULT and ON UPDATE in the column definition to get automatic initialization and updating to the current date and time
TIMESTAMP(<i>fsp</i>)	A timestamp. TIMESTAMP values are stored as the number of seconds since the Unix epoch ('1970-01-01 00:00:00' UTC). Format: YYYY-MM-DD hh:mm:ss. The supported range is from '1970-01-01 00:00:01' UTC to '2038-01-09 03:14:07' UTC. Automatic initialization and updating to the current date and time can be specified using DEFAULT CURRENT_TIMESTAMP and ON UPDATE CURRENT_TIMESTAMP in the column definition
TIME(<i>fsp</i>)	A time. Format: hh:mm:ss. The supported range is from '-838:59:59' to '838:59:59'
YEAR	A year in four-digit format. Values allowed in four-digit format: 1901 to 2155, and 0000. MySQL 8.0 does not support year in two-digit format.



DDL(Data Definition Language)

- DDL is short name of Data Definition Language, which deals with database schemas and descriptions, of how the data should reside in the database.
- CREATE - to create a database and its objects like (table, index, views, store procedure, function, and triggers)
- ALTER - alters the structure of the existing database
- DROP - delete objects from the database
- TRUNCATE - remove all records from a table, including all spaces allocated for the records are removed
- RENAME - rename an object

Rules for naming a database object or table-columns

1. Table should start with an alphabet; it can contain minimum 1 character and maximum 30 character. It should not allow any space or any other special character except `_`, `#`, `$` and 0 to 9
2. A table can have minimum '1' column and maximum 1000 columns
3. All the oracle reserved / keyword should not be used as a column names or table names
4. A table can have minimum '0' records and maximum 'n' number of records upto hard disk capacity
5. The rules which we are following for the table names the same rules are applicable for column name

CREATE Statement

- This command is used to create any database object such as database, table, view, sequence, index etc.

Syntax: CREATE DATABASE DATABASENAME;

Example: CREATE DATABASE kmitcsma;

Use the SHOW statement to find out what databases currently exist on the server:

Syntax: SHOW DATABASES;

We have to select the database on which we want to run the database queries.

Syntax: USE DATABASE_NAME;

Example: USE kmitcsma;

The SQL CREATE TABLE Statement

The CREATE TABLE statement is used to create a new table in a database.

Syntax:

```
CREATE TABLE table_name (  
column1 datatype(size),  
column2 datatype(size),  
column3 datatype(size),  
....  
);
```

Example: create table emp (eid int(5), ename varchar(15));

DESC: This SQL command is used to show the structure of the table

Syntax: desc tablename;

Example: desc emp;

ALTER COMMAND

ALTER: This command is used to modify the structure of table. Using this command, we can perform 4 operations on table they are:

- **ALTER-MODIFY**
- **ALTER-ADD (Columns or Constraints)**
- **ALTER-DROP (Columns or Constraints)**
- **ALTER-RENAME**

❖ **ALTER-MODIFY**: This command is used to increase or decrease the size of the data type and also change the data type from old data type to new data type.

Syntax: ALTER TABLE tablename MODIFY old_columnname new_datatype(size);

Example: alter table emp modify ename varchar(25);

Syntax to modify more than one column:

Alter table tablename modify(columnname-1 datatype(size),columnname-2 datatype(size),.....,columnname-n datatype(size));

ALTER COMMAND

ALTER-ADD: This command is used to add the new column to the existing table.

Syntax: ALTER TABLE tablename ADD(columnname-1 datatype(size),columnname-2 datatype(size),.....,columnname-n datatype(size));

Example: alter table emp add(dob date, mobilen no int(10));

Constraints can be added using ALTER command:

ALTER TABLE emp

Add primary key (eid);

ALTER-DROP: This command is used to remove the columns from the existing table.

Syntax: ALTER TABLE tablename DROP column columnname;

Example: alter table employee drop column mobilen no;

ALTER COMMAND

ALTER-RENAME: This command is used to Changes the name of an existing column from the existing table.

Syntax:

```
ALTER TABLE table_name
```

```
RENAME COLUMN old_column_name TO new_column_name datatype;
```

OR

```
ALTER TABLE table_name
```

```
CHANGE COLUMN old_column_name new_column_name datatype [constraints];
```

Example:

```
ALTER TABLE emp
```

```
RENAME COLUMN eid TO empno;
```

DROP and TRUNCATE

- Drops commands remove tables and databases from RDBMS.

To drop a Table

Syntax: drop table table_name;

Ex: drop table emp;

- To Drop a database

syntax: Drop database database_name;

Ex: Drop database kmitcsma;

TRUNCATE

The major definition between TRUNCATE and DROP is that truncate is used to delete the data inside the table not the whole table.

Syntax: TRUNCATE TABLE table_name;

Ex: Truncate table emp;

RENAME table

Syntax: Renaming the existing table:

```
RENAME table table_name to new table_name;
```

Ex:

```
mysql> rename table persons to emp;
```

Comments in SQL

- **Comments** are used to explain sections of SQL statements.

Comments can be written in the following three formats:

1. Single-line comments
2. Multi-line comments
3. In-line comments

Comments in SQL

1.Single Line Comments

- Comments starting and ending in a single line are considered single-line comments. A line starting with ‘- -’ is a comment and will not be executed.

Ex: create database CSMA; --THIS IS SINGLELINECOMMENT:

2.Multi-Line Comments

- Comments starting in one line and ending in different lines are considered as multi-line comments.
- A line starting with ‘/*’ is considered as starting point of the comment and is terminated when ‘*/’ is encountered.

**Ex: /*This is for multiline comments
specifying the database is created:*/**

Create database kmitCSMA;

In-line comments

3.In-Line Comments

- In-line comments are an extension of multi-line comments; comments can be stated in between the statements and are enclosed in between ‘/*’ and ‘*/’.

SYNTAX:

SELECT column name

/* This column contains the name of the customer / order_date /

This column contains the date the order was placed */

FROM tablename;

EX: SELECT ename

/* This column contains the name of the customer / order_date /

This column contains the date the order was placed */

FROM emp;

DML(Data Manipulation Language)

- DML stands for **Data Manipulation Language**. The schema (Table) created by DDL (Data Definition Language) is populated or filled using Data Manipulation Language.
- DDL fill the rows of the table, and each row is called **Tuple**.
- Using DML, you can insert, modify, delete and retrieve the information from the Table.

DML(**Data Manipulation Language**)

DML is short name of **Data Manipulation Language** which deals with data manipulation and includes most common SQL statements such SELECT, INSERT, UPDATE, DELETE, etc., and it is used to store, modify, retrieve, delete and update data in a database.

- INSERT - insert data into a table
- SELECT - retrieve data from a database or database objects
- UPDATE - updates existing data within a table
- DELETE - Delete all records from a database table

Inserting the tuples/records into the table.

Two (2) ways to insert the rows into the table.

1. By specifying column names
2. Without specifying column names.

1. Syntax:

INSERT INTO table_name (column1, column2, column3....)

VALUES (value1, value2, value3.....);

Ex: Insert into emp(eid, ename, eadd, dob, mobileno) values (100,'rama',;Hyderabad','2023-04-20');

Inserting data into table Without specifying column names

2. Syntax:

```
INSERT INTO table_name  
VALUES (value1, value2, value3....);
```

```
Ex: Insert into emp values (100,'rama',;Hyderabad','2023-04-20');
```

Inserting multiple rows into the table.

Syntax:

```
INSERT INTO TABLE_NAME (col1, col2, col3,... col N) VALUES  
(value1, value2, value3, .... valueN),  
(value1, value2, value3, .... valueN),  
.....  
(value1, value2, value3, .... valueN);
```

```
Insert into emp(eid, ename, eadd, dob, mobilenos) values  
(100,'rama',;Hyderabad','2023-04-20'),  
( 101, 'krishna', 'Hyderabad', '2023-03-29')  
( 102, 'srikar', 'Atlanta', '2022-08-19')  
( 103, 'srinidhi', 'USA', '2005-02-19');
```

SELECT command

The **SELECT** statement is used to select data from a database. The data returned is stored in a result table, called the result-set.

Syntax to select selected columns and all rows:

```
SELECT column1, column2, ...  
FROM table_name;
```

Ex: select eid,ename from emp;

Here, column1, column2, ... are the field names of the table you want to select data from.

If you want to select all the fields available in the table, use the following syntax:

Syntax to select all rows and columns:

```
Syntax: SELECT * FROM table_name;
```

```
Ex: Select * from emp;
```

The SQL WHERE Clause

- The WHERE clause is used to filter records.
- It is used to extract only those records that fulfill a specified condition.
- SYNTAX: `SELECT column1, column2, ...
FROM table name
WHERE condition;`

ex: select * from emp
where eadd='hyderabad';
OR

select eid, ename address from emp
where eid= 102;

UPDATE command

- UPDATE command is used to modify the data in the table.
- Using this command we can modify all the records in the table and also we can modify specific records in the table.

Syntax to update all records in a single column:

Syntax: UPDATE tablename SET col-1=value1, col-2=value2..;

Example: update employee set ename= 'sachin';

- Syntax to update a particular record (using where clause):

Syntax: UPDATE tablename set column name=value where condition;

- Example: update employee set deptno=20 where eid=101;

1. Update one column

```
UPDATE Student SET City = 'Secunderabad' WHERE City = 'Hyderabad';
```

2. Update multiple columns

```
UPDATE Courses  
SET CourseName = 'Advanced DBMS',  
    Credits = 5 WHERE CourseID = 101;
```

3. Update all rows (be careful — no WHERE clause)

```
UPDATE Student  
SET City = 'Unknown';
```

DELETE command

DELETE command is used to delete the records from the existing table. Using this command we can delete all records and also specific record in the table (using where clause).

- Syntax for deleting a particular record:

Syntax: DELETE from tablename where condition;

- Example: delete from employee where eid=102;

- Syntax for deleting all records:

Syntax: DELETE FROM tablename;

Example: delete from employee;

Create Table Using Another Table (On-the-fly Table Creation)

- A copy of an existing table can also be created using CREATE TABLE.
- The new table gets the same column definitions. All columns or specific columns can be selected.
- If you create a new table using an existing table using **AS** keyword, the new table will be filled with the existing values from the old table but constraint were not copied.

Syntax:

```
CREATE TABLE new_table_name AS SELECT column1, column2,...  
FROM existing_table_name  
WHERE ....;
```

Example:

```
CREATE TABLE TestTable AS  
SELECT customername, contactname  
FROM customers;
```

INTEGRITY CONSTRAINTS OVER RELATIONS

- An integrity constraint (IC) is a condition specified on a database schema and restricts the data that can be stored in an instance of the database.
- ICs are specified when schema is defined by DBA or end user.
- IC's are used to ensure accuracy and consistency of the data in relational database.
- ICs are checked when relations are modified.
- IC's are set of rules that database is not permitted to violate.
- If a database instance satisfies all the integrity constraints specified on the database schema, it is a **legal** instance.
 - DBMS should not allow illegal instances.
- If the DBMS checks ICs, stored data is more faithful to real-world meaning.
 - Avoids data entry errors, too!
 - **IC's guard against accidental damage to the database.**

Types of integrity constraints

1. Domain Integrity
2. Entity Integrity Constraint
3. Referential Integrity Constraint
4. Key Constraints

Domain Integrity

- Domain integrity means the definition of a valid set of values for an attribute.
- The data type of domain includes string, character, integer, time, date, currency, etc. The value of the attribute must be available in the corresponding domain.
 - data type,
 - length or size,
 - is null value allowed,
 - is the value unique or not for an attribute,
 - the default value,
 - the range (values in between) and/or
 - specific values for the attribute) and/or
 - specific values for the attribute.
- Domain Constraints are user-defined columns that help the user to enter the value according to the data type.
- And if it encounters a wrong input it gives the message to the user that the column is not fulfilled properly.
- Or in other words, it is an attribute that specifies all the possible values that the attribute can hold like integer, character, date, time, string, etc.
- It defines the domain or the set of values for an attribute and ensures that the value taken by the attribute must be an atomic value(Can't be divided) from its domain.

Here Emp id 102 , location 4578 is not allowed because location is character attribute.

Eid	Ename	Job	Sal	Location	Deptno
100	Rama	Scientist	102000	Hyd	12
101	Srikar	Manager	80300	Hyd	20
102	Krishana	Analyst	95600	4578	24

Entity Integrity

Constraint-

- This rule states that in any database relation value of attribute of a primary key **can't be null.**
- This is because the primary key value is used to identify individual rows in relation and if the primary key has a null value, then we can't identify those rows.
- A table can contain a null value other than the primary key field.

EMPLOYEE

EMP_ID	EMP_NAME	SALARY
123	Jack	30000
142	Harry	60000
164	John	20000
	Jackson	27000

Not allowed as primary key can't contain a NULL value

- Ex- Consider a relation "EMPLOYEE" Where "Emp_id" is a primary key and it must not contain any null value whereas other attributes may contain null value

Referential Integrity

Constraint

- Referential integrity Specifies that rows cannot be deleted, which are used by other records.
- A referential integrity constraint is specified between two tables.
- The foreign key a constraint is a column or list of columns that points to the primary key or unique key column of another table
- In the Referential integrity constraints, if a foreign key in Table 1 refers to the Primary Key or unique key of Table 2, then every value of the Foreign Key in Table 1 must be null or be available in Table 2.
- The main purpose of the foreign key is only those values are allowed in the present table that will match the primary key column of another table
- The rules are:
 - can't delete a record from a primary table if matching records exist in a related table.
 - can't change a primary key value in the primary table if that record has related records.
 - can't enter a value in the foreign key field of the related table that doesn't exist in the primary key of the primary table.
 - However, you can enter a Null value in the foreign key, specifying that the records are unrelated.

Referential Integrity Constraint-Example:

Example:

**EMP-
CHILD TABLE**

(Table 1)

EMP_NAME	NAME	AGE	D_No
1	Jack	20	11
2	Harry	40	24
3	John	27	18
4	Devil	38	13

Foreign key

Not allowed as D_No 18 is not defined as a Primary key of table 2 and In table 1, D_No is a foreign key defined

Relationships

(Table 2)

Primary Key

<u>D_No</u>	D_Location
11	Mumbai
24	Delhi
13	Noida

**DEPT---
PARENT TABLE**

Examples

Rule 1. You can't delete any of the rows in the "DEPT" relation that are visible since all the "DEPT" are in use in the "EMP" relation.

Rule 2. You can't change any of the "D_NO" in the "DEPT" relation since all the "D_NO" are in use in the "EMP" relation.

Rule 3. The values that you can enter in the "D_NO" field in the "EMP" relation must be in the "DEPT" relation.

Rule 4. You can enter a null value in the "EMP" relation if the records are unrelated.

Note : Name of foreign key can be different than the name of primary key it represent in other table. For example in our Employee and Department relationship, Primary key in Department table is d_no and we have used same name in Employee table to create foreign key.

It could have been different e.g. deptno or departmentID etc.

Key Constraints

- Keys are the entity set that is used to identify an entity within its entity set uniquely.
- A key constraint is a statement that a certain minimal subset of the fields of a relation is a unique identifier for a tuple.
- An entity set can have multiple keys, but out of which one key will be the primary key. A primary key can contain a unique and null value in the relational table.
- Below example ID is primary key but ID have the same value 1002. so, it is violating the key constraint

ID	NAME	SEMENSTER	AGE
1000	Tom	1 st	17
1001	Johnson	2 nd	24
1002	Leonardo	5 th	21
1003	Kate	3 rd	19
1002	Morgan	8 th	22

Not allowed. Because all row must be unique

Other Types of Constraints

- SQL Constraints are the rules applied to a data columns or the complete table to limit the type of data that can go into a table. When you try to perform any INSERT, UPDATE, or DELETE operation on the table, RDBMS will check whether that data violates any existing constraints
 - NOT NULL constraint
 - PRIMARY KEY constraint
 - FOREIGN KEY constraint
 - UNIQUE constraint
 - CHECK constraint
 - DEFAULT Constraint
 - AUTO_INCREMENT

NOT NULL Constraint

The NOT NULL constraint in SQL is used to ensure that a column in a table doesn't contain NULL (empty) values, and prevent any attempts to insert or update rows with NULL values.

NOT NULL Ensures that a column cannot have a NULL value.

The NOT NULL constraint enforces a column to NOT accept NULL values, which means that you cannot insert a new record, or update a record without adding a value to this field.

NULL represents a record where data may be missing data or data for that record may be optional

Once not null is applied to a particular column, you cannot enter null values to that column and restricted to maintain only some proper value other than null

A not-null constraint cannot be applied at table level

NOT NULL Constraint

NOT NULL on CREATE TABLE

CREATE TABLE Persons (

PID int (3) NOT NULL,

PName varchar (10) NOT NULL,

PADD varchar (20) NOT NULL,

PAge int (2)

);

NOT NULL Constraint

- By default, a column can hold NULL values

The term NULL in SQL is used to specify that a data value does not exist in the database. It is not the same as an empty string or a value of zero, and it signifies the absence of a value or the unknown value of a data field.

Some common reasons why a value may be NULL –

- The value may not be provided during the data entry.
- The value is not yet known.

It is important to understand that you cannot use comparison operators such as “=”, “<”, or “>” with NULL values. This is because the NULL values are unknown and could represent any value. Instead, you must use “IS NULL” or “IS NOT NULL” operators to check if a value is NULL

Syntax: The basic syntax of NULL while creating a table.

```
SQL> CREATE TABLE CUSTOMERS (  
    ID INT(3) NOT NULL,  
    NAME VARCHAR (20) NOT NULL,  
    AGE INT(2) NOT NULL,  
    ADDRESS VARCHAR (25) ,  
    SALARY FLOAT(6, 2)  
);
```

In the above example SALARY and ADDRESS are NULL

PRIMARY KEY constraint

The PRIMARY KEY constraint uniquely identifies each record in a table.

Primary keys must contain UNIQUE values, and cannot contain NULL values.

A table can have only **ONE primary key**; and in the table, this primary key can consist of single or multiple columns (fields).

```
CREATE TABLE Persons (  
    PID int (3),  
    PName varchar (10) NOT NULL,  
    PADD varchar (20) NOT NULL,  
    PAge int (2),  
    PRIMARY KEY (PID)  
);
```


FOREIGN KEY Constraint

- The FOREIGN KEY constraint is used to prevent actions that would destroy links between tables.
- A FOREIGN KEY is a field (or collection of fields) in one table, that refers to the PRIMARY KEY or UNIQUE column in another table.
- The table with the foreign key is called the child table, and the table with the primary key is called the referenced or parent table.

FOREIGN KEY Constraint

```
CREATE TABLE dept
(
deptno int(3), dname varchar(20), location varchar(20),
Primary key(deptno)
); -- PARENT TABLE
```

```
CREATE TABLE Emp
( Eid int(3), ename varchar(20), salary float(6,2), dno int(3),
PRIMARY KEY (Eid),
FOREIGN KEY (dno) REFERENCES dept(deptno)) ; -- CHILD TABLE
```

```
mysql> CREATE TABLE Emp
-> ( Eid int(3), ename varchar(20), salary float(6,2), dno int(3),
-> PRIMARY KEY (Eid),
-> FOREIGN KEY (dno) REFERENCES dept(deptno));
Query OK, 0 rows affected, 3 warnings (0.46 sec)
```

Name of foreign key can be different than the name of primary key it represent in other table. But datatype should be same otherwise Referencing column 'dno' and referenced column 'deptno' in foreign key constraint 'emp_ibfk_1' are incompatible.

Difference between Primary key and Foreign key in SQL

- Here is some important difference between primary and foreign keys in a table which is worth remembering both on SQL interview point of view and knowledge point of view.
 - 1) Name of foreign key can be different than the name of primary key it represent in other table. For example in our Employee and Department relationship, Primary key in Department table is dept_id and we have used same name in Employee table to create foreign key. It could have been different e.g. departmentId or departmentID etc.
 - 2) Another difference between primary and foreign key is that unlike primary key, a foreign key can be null e.g. in our example you can have an Employee record for which dept_id can be null, this shows that no corresponding record in Department table.
 - 3) One more difference between primary key and foreign key is that foreign key can be duplicate opposite to primary key which is always unique.
 - 4) By using foreign key constraints, we can introduce referential integrity in multiple table relationship in SQL. Referential integrity guarantees data integrity, see benefits of Referential Integrity in SQL to know more.
 - 5) Foreign key mostly works as a link between two table when we join tables using INNER JOIN and OUTER JOIN. For example, when we INNER JOIN both Employee with Department table, we can use dept_id as joining column. See How to join three tables in SQL for more details.
 - 6) Table on which a column is declared as a primary key is known as parent table in the relationship and foreign key table is known as child table in a relationship. For example in Employee and Department relationship, Department is parent table because dept_id is primary key there and Employee is child table because dept_id is a foreign key in this table

UNIQUE Constraint

The UNIQUE constraint ensures that all values in a column are different.

Both the UNIQUE and PRIMARY KEY constraints provide a guarantee for uniqueness for a column or set of columns.

A PRIMARY KEY constraint automatically has a UNIQUE constraint.

Difference between UNIQUE constraints and PRIMARY KEY constraint is unique allows NULL values where as primary key not allowed the NULL values.

```
CREATE TABLE Persons (  
    PID int (3) UNIQUE,  
    PName varchar (10) NOT NULL,  
    PADD varchar (20),  
    PAge int (2),  
    Padhar varchar(12)  
    PRIMARY KEY (Padhar)  
);
```

In this table we can enter NULL values for PID where as
for Padhar
column we can not enter NULL Values

CHECK constraint

The CHECK constraint is used to limit the value range that can be placed in a column using predefined condition.

If you define a CHECK constraint on a column it will allow only certain values for this column.

The CHECK constraint comes into action to evaluate the inserted or modified values, where the value that satisfies the condition will be inserted into the table, otherwise, the insert operation will be discarded.

It is allowed to specify multiple CHECK constraints for the same column

The following SQL creates a CHECK constraint on the "Age" column when the "Persons" table is created. The CHECK constraint ensures that the age of a person must be 18, or older otherwise shows CHECK Constraint Violating.

```
CREATE TABLE Persons (  
    PID int (3) ,  
    PName varchar (10) NOT NULL,  
    PAdd varchar (20) NOT NULL,  
    PAge int(2),  
    PRIMARY KEY(PID),  
    CHECK (PAge>=18)  
);
```

DEFAULT

- The DEFAULT constraint is used to provide a **default value** for a column when no value is specified during INSERT.

```
CREATE TABLE table_name (  
    column_name datatype DEFAULT default_value  
);
```

AUTO_INCREMENT

```
CREATE TABLE person (  
    pid INT AUTO_INCREMENT PRIMARY KEY,  
    page INT CHECK (page >= 18)  
);
```

Note: By default, the number(pid) will start from value 1.

Set the starting value

If you want numbering to begin from a specific value (say 1000):

```
SQL>ALTER TABLE person AUTO_INCREMENT = 1000;
```

AUTO_INCREMENT

```
CREATE TABLE p1 (  
    pid INT AUTO_INCREMENT PRIMARY KEY,  
    page INT CHECK (page >= 18)  
) AUTO_INCREMENT =100;
```

Note: The number will begin from 100

Add a Primary Key

- Adds a primary key constraint to an existing column.

Syntax:

```
ALTER TABLE table_name  
ADD PRIMARY KEY (column_name);
```

Example:

```
ALTER TABLE employees  
ADD PRIMARY KEY (id);
```

Drop a Primary Key

- Removes the primary key constraint from the table.

Syntax:

```
ALTER TABLE table_name  
DROP PRIMARY KEY;
```

Example:

```
ALTER TABLE table_name  
DROP PRIMARY KEY;
```

Add a Foreign Key

- Adds a foreign key constraint to link the table to another table.

Syntax:

```
ALTER TABLE table_name
```

```
ADD CONSTRAINT constraint_name
```

```
FOREIGN KEY (column_name) REFERENCES parent_table (parent_column);
```

Example: Assume a departments table exists with dept_id as its primary key. Add a foreign key to the employees table.

```
ALTER TABLE employees
```

```
ADD CONSTRAINT fk_dept FOREIGN KEY (department_id) REFERENCES  
departments(dept_id);
```

Drop a Foreign Key

- Removes a foreign key constraint.

Syntax:

```
ALTER TABLE table_name  
DROP FOREIGN KEY constraint_name;
```

Example:

```
ALTER TABLE employees  
DROP FOREIGN KEY fk_dept;
```

ENFORCING INTEGRITY CONSTRAINTS

Consider Students and Enrolled;

- CREATE TABLE Students (sid VARCHAR(20) , name VARCHAR (30) , login VARCHAR(20) , age INT, gpa FLOAT(2,2),PRIMARY KEY (sid));
- CREATE TABLE Enrolled (
studid VARCHAR(20) ,
cid VARCHAR(20), grade VARCHAR(10),
PRIMARY KEY (studid, cid),
FOREIGN KEY (studid) REFERENCES Students(sid)) ;
- The foreign key constraint states that every st'udid value in Enrolled must also appear in Students, that is, studid in Enrolled is a foreign key referencing Students. Specifically, every studid value in Enrolled must appear as the value in the primary key field, sid, of Students

SQL provides several alternative ways to handle foreign key violations. We must consider three basic questions

1. What should we do if an Enrolled row is inserted, with a studid column value that does not appear in any row of the Students table?

In this case, the INSERT command is simply rejected.

2. What should we do if a Students row is deleted?

The options are:

- Delete all Enrolled rows that refer to the deleted Students row.
- Disallow the deletion of the Students row if an Enrolled row refers to it.
- Set the studid column to the sid of some (existing) 'default' student, for every Enrolled row that refers to the deleted Students row
- For every Enrolled row that refers to it, set the studid column to null. In our example, this option conflicts with the fact that stud'id is part of the primary key of Enrolled and therefore cannot be set to null. Therefore, we are limited to the first three options in our example, although this fourth option (setting the foreign key to null) is available in general.

SQL provides several alternative ways to handle foreign key violations. We must consider three basic questions

3.What should we do if the primary key value of a Students row is updated?

The options here are similar to the previous case

SQL allows us to choose any of the four options on DELETE and UPDATE. For example, we can specify that when a Students row is deleted, all Enrolled rows that refer to it are to be deleted as well, but that when the sid column of a Students row is modified, this update is to be rejected if an Enrolled row refers to the modified Students row:

```
CREATE TABLE Enrolled (  
    studid VARCHAR(20) , cid VARCHAR(20) , grade VARCHAR(10),  
    PRIMARY KEY (studid, dd),  
    FOREIGN KEY (studid) REFERENCES Students ON DELETE CASCADE ON UPDATE NO ACTION  
);
```

1.ON DELETE CASCADE: When we create a foreign key using this option, that if the parent record is deleted, any child records are also deleted. ...

2.ON UPDATE CASCADE: means that if the parent primary key is changed, the child value will also change to reflect that.

- The default option is NO ACTION, which means that the action (DELETE or UPDATE) is to be rejected.
- Thus, the ON UPDATE clause in our example could be omitted, with the same effect.
- The CASCADE keyword says that, if a Students row is deleted, all Enrolled rows that refer to it are to be deleted as well.
- If the UPDATE clause specified CASCADE, and the *sid* column of a Students row is updated, this update is also carried out in each Enrolled row that refers to the updated Students row.
- If a Students row is deleted, we can switch the enrollment to a 'default' student by using ON DELETE SET DEFAULT.
- It is really not appropriate to switch enrollments to a default student.
- The correct solution in this example is to also delete all enrollment tuples for the deleted student (that is, CASCADE) or to reject the update.
- SQL also allows the use of *null* as the default value by specifying ON DELETE SET NULL.

Transactions and Constraints

- A program that runs against a database is called a transaction, and it can contain several statements (queries, inserts, updates, etc.) that access the database.
- If (the execution of) a statement in a transaction violates an integrity constraint, should the DBMS detect this right away or should all constraints be checked together just before the transaction completes?
- By default, a constraint is checked at the end of every SQL statement that could lead to a violation, and if there is a violation, the statement is rejected.
- Sometimes this approach is too inflexible.
- Consider the following variants of the Students and Courses relations; every student is required to have an honors course, and every course is required to have a grader, who is some student.

```
CREATE TABLE Students (  
    sid VARCHAR(20) ,  
    name VARCHAR(30),  
    login VARCHAR (20) ,  
    age INT,  
    honors VARCHAR(10) NOT NULL,  
    gpa FLOAT(2,2) PRIMARY KEY (sid),  
    FOREIGN KEY (honors) REFERENCES Courses (cid)  
);  
  
CREATE TABLE Courses (  
    cid VARCHAR(10),  
    cname VARCHAR ( 10) ,  
    credits INT,  
    grader VARCHAR(20) NOT NULL,  
    PRIMARY KEY (cid)  
    FOREIGN KEY (grader) REFERENCES Students (sid)  
);
```

- Whenever a Students tuple is inserted, a check is made to see if the "honors course is in the Courses relation, and whenever a Courses tuple is inserted, a check is made to see that the grader is in the Students relation.
- How are we to insert the very first course or student tuple?
- SQL allows a constraint to be in DEFERRED or IMMEDIATE mode.
- **INITIALLY IMMEDIATE** means that the constraint is checked whenever you add, update, or delete rows from a table. **INITIALLY DEFERRED** means that the constraint is only checked when a transaction is committed.

SET CONSTRAINT ConstraintFoo DEFERRED

- A constraint in deferred mode is checked at commit time.
- In our example, the foreign key constraints on Students and Courses can both be declared to be in deferred mode.
- We can then insert Students with a nonexistent Courses (temporarily making the database inconsistent), insert the Courses (restoring consistency), then commit and check that both constraints are satisfied.

QUERYING RELATIONAL DATA

DBMS -UNIT-2

QUERYING RELATIONAL DATA

- Relational data means information stored in tables with relationships (via primary keys, foreign keys, and constraints).
- Querying is done using SQL (Structured Query Language).
- A relational database query (query, for short) is a question about the data and the answer consists of a new relation containing the result.

Basic Query Structure:

```
SELECT column1, column2, ...  
FROM table_name  
WHERE condition  
GROUP BY column  
HAVING group_condition  
ORDER BY column [ASC|DESC]  
LIMIT n;
```

QUERYING RELATIONAL DATA

A relational database query (query, for short) is a question about the data and the answer consists of a new relation containing the result.

- What does the following query compute?

SELECT S.name, E.cid

FROM Students as S, Enrolled as E

WHERE S.sid=E.sid;

Given the following instances of Enrolled and Students:

- We get:

S.name	E.cid
Smith	Topology112
Jones	History105

sid	name	login	age	gpa
53666	Jones	jones@cs	18	3.4
53688	Smith	smith@eecs	18	3.2
53650	Smith	smith@math	19	3.8

sid	cid	grade
53831	Carnatic101	C
53831	Reggae203	B
53650	Topology112	A
53666	History105	B

SQL Operators

Arithmetic

- ADD (+)
- SUBTRACT (-)
- MULTIPLY (*)
- DIVIDE (/) — Quotient
- MODULO (%) — Remainder

Calculations on
Numerical data types

Addition (+)  Sum(Sales)

Comparison

- Equal to (=)
- Greater than (>) (>=)
- Less than (<) (<=)
- NOT (!=)

Compare data
values

Ex:
Customers who live in a specific state
Orders placed after a certain date
✓ Essential for filtering relevant data

Logical

- AND (TRUE - if all are true)
- OR (TRUE if at least one is true)
- NOT (Negates a condition)
TRUE - if the condition is false.

Combine multiple conditions
within your queries

CONDITIONAL SELECTIONS AND OPERATORS

We have two clauses used in this

Where

Order by

Where Clause:

The **WHERE** clause in SQL is used to filter records in a table.

It specifies a **condition** that must be true for the rows to be included in the result

```
SELECT column1, column2, ...
```

```
FROM table_name
```

```
WHERE condition;
```


The following are the different types of operators used in where clause.

- ❖ **Arithmetic operators**
- ❖ **Comparison operators**
- ❖ **Logical operators**

Arithmetic operators

-- highest precedence

+, -, *, /

- **Comparison operators**

- =, !=, >, <, >=, <=, <>
- between, not between
- in, not in
- is null, is not null
- like, not like

Logical operators

❖ **And**

❖ **Or**

❖ **Not**

-- lowest precedence

using =, >, <, >=, <=, !=, <>

- **SQL> select * from student where Sno = 2;**
- **SQL> select * from student where Sno < 2;**
-
- **SQL> select * from student where Sno > 2;**
- **SQL> select * from student where Sno <= 2;**
- **SQL> select * from student where Sno != 2;**

using AND

- This will gives the output when all the conditions become true.

Syntax:

- `select * from <table_name> where <condition1> and <condition2> and .. <conditionn>;`

SQL> select * from student where sno = 2 and marks >= 87;

using OR

This will gives the output when either of the conditions become true.

Syntax:

```
select * from <table_name> where <condition1> and  
<condition2> or .. <conditionn>;
```

Ex:

```
SQL> select * from student where Sno = 4 or marks >= 100;
```

using BETWEEN

This will gives the output based on the column and its lower bound, upperbound.

Syntax:

select * from *<table_name>* where *<col>* between *<lower bound>* and *<upper bound>*;

Ex:

SQL> select * from student where marks between 70 and 98;

using NOT BETWEEN

This will gives the output based on the column which values are not in its lower bound , upperbound

Syntax:

```
select * from <table_name> where <col> not between  
<lower bound> and <upper bound>;
```

example

```
SQL> select * from student where marks not between 80 and  
90;
```

using IN

This will gives the output based on the column and its list of values specified.

Syntax:

```
select * from <table_name> where <col> in ( value1,  
value2, value3 ... valuen);
```

Ex:

```
SQL> select * from student where Sno in (1, 2, 3);
```


using NOT IN

This will gives the output based on the column which values are not in the list of values specified.

Syntax:

```
select * from <table_name> where <col> not in ( value1,  
value2, value3 ... valuen);
```

Ex:

```
SQL> select * from student where no not in (1, 2, 3);
```

using IS NULL

This will gives the output based on the null values in the specified column.

Syntax:

```
select * from <table_name> where <col> is null;
```

Ex:

```
SQL> select * from student where marks is null;
```

using **IS NOT NULL**

This will gives the output based on the not null values in the specified column.

Syntax:

select * from <table_name> where <col> **is not null;**

Ex:

SQL> select * from student where marks **is not null;**

using LIKE

- The LIKE operator is used in a WHERE clause to search for a pattern in a column's value.
- It is often used with wildcards.
- This will be used to search through the rows of database column based on the pattern you specify

Syntax:

```
select * from <table_name> where <col> like <pattern>
```

Wildcards in LIKE:

% → Matches zero or more characters

_ → Matches exactly one character

ii) This will give the rows whose name start with 'S'.

SQL> select * from student where Name like 'S%';

iii) This will give the rows whose name ends with 'h'.

SQL> select * from student where name like '%a';

iV) This will give the rows whose name's second letter start with 'a'.

SQL> select * from student where name like '_a%';

v) This will give the rows whose name's third letter start with 'd'.

SQL> select * from student where name like '__h%';

vi) This will give the rows whose name's second letter start with 't' from ending.

SQL> select * from student where name like '%t_';

vii) This will give the rows whose name's third letter start with 'e' from ending.

SQL> select * from student where name like '%e__';

viii) This will give the rows whose name contains 2 a's.

SQL> select * from student where name like '%a% a %';

Not matching a pattern using **NOT LIKE**:

```
SELECT ename FROM employees  
WHERE ename NOT LIKE '%a%';
```

Matches employees whose name does not contain 'a'.

using ORDER BY

- This will be used to ordering the columns data (ascending or descending).

Syntax:

Select * from <table_name> order by <col> desc;

- If you want output in descending order you have to use *desc* keyword after the column.

Ex:

SQL> select * from student order by no;

SQL> select * from student order by no desc;

using **LIMIT**

- The LIMIT clause specifies the maximum number of rows(Top) to retrieve from the query result.

Syntax1:

Select * from <table_name> LIMIT n;

//This query returns only the first n rows from the table.

Ex1: SQL> SELECT * FROM products LIMIT 3;

//This query returns only 3 rows for a quick preview of the products table

Ex2: Fetch Top 3 Products by Price:

SQL> SELECT product_name, price FROM products ORDER BY price DESC LIMIT 3;

using LIMIT

- The LIMIT clause specifies the maximum number of rows(Top) to retrieve from the query result.

Syntax2:

Select * from <table_name> LIMIT m , n;

//The query skips the first m rows and returns the next n rows from the result set.

Syntax3:

Select * from <table_name> LIMIT m offset n;

//The query skips the first n rows and returns the next m rows from the result set.

AGGREGATE OPERATORS (Functions)

DBMS -UNIT-2

Aggregate Operators(Functions)

- In addition to simply retrieving data, we often want to perform some **computation** or **summarization**.
- SQL allows a powerful class of constructs for **computing aggregate values** such as COUNT, MIN,MAX,SUM etc.
- Aggregate functions are used in MySQL **to calculate a set of values and return a single value as a result**.
- These functions are useful when we want to analyze data stored in a table and **generate summary information**, such as the total number of rows, the sum of values in a column, or the average of a set of values.
- Using aggregate functions, we can **filter data, calculate statistics, and improve performance** (by limiting the data that needs to be processed).

- SQL supports **aggregate operations**, which can be applied on any column, say A, of a relation:
- 1. COUNT ([DISTINCT] A): The number of (unique) values in the A column.
- 2. SUM ([DISTINCT] A): The sum of all (unique) values in the A column.
- 3. AVG ([DISTINCT] A): The average of all (unique) values in the A column.
- 4. MAX(A) : The Maximum value in the A Column.
- 5. MIN(A) : The Minimum value in the A Column,

GROUP BY Clause

- The GROUP BY clause is often used with aggregate functions (MAX, SUM, AVG) to group the results by one or more columns.
- In simple words we can say that The GROUP BY clause is used in collaboration with the SELECT statement to arrange required data into groups.
- The GROUP BY statement groups rows that have the same values. This Statement is used after the where clause. This statement is often used with some aggregate function like SUM, AVG, COUNT etc. to group the results by one or more columns.

Syntax: *SELECT* **column1**, *AGG_FUNC(column2)*
 FROM table_name
 GROUP BY column1;

HAVING Clause

- The **HAVING clause** is used to **filter groups** created by the **GROUP BY clause**.
- It applies conditions on **aggregated data** and is typically used with **aggregate_functions** like SUM, AVG, or COUNT.
- Unlike the **WHERE clause**, which filters rows before grouping, the HAVING clause filters groups after the grouping process.

Key Features of the HAVING Clause:

- Filters groups based on conditions involving aggregate functions.
- Always used after the GROUP BY clause.
- Works with aggregate functions such as SUM, COUNT, AVG, MIN, MAX.



HAVING Clause

Syntax:

```
SELECT column name(s) FROM table name  
WHERE condition  
GROUP BY column_name(s) HAVING condition  
ORDER BY column_name(s);
```

WHERE vs HAVING

WHERE → filters rows before grouping.

HAVING → filters groups after grouping.

So:

WHERE works on **individual rows**.

HAVING works on **aggregate results (SUM, AVG, COUNT, etc.)**.

EMP TABLE

empno	ename	job	mgr	hiredate	sal	comm	deptno
7369	SMITH	CLERK	7902	1980-12-17	800	NULL	20
7499	ALLEN	SALESMAN	7698	1981-02-20	1600	300	30
7521	WARD	SALESMAN	7698	1981-02-22	1250	500	30
7566	JONES	MANAGER	7839	1981-04-02	2975	NULL	20
7654	MARTIN	SALESMAN	7698	1981-09-28	1250	1400	30
7698	BLAKE	MANAGER	7839	1981-05-01	2850	NULL	30
7782	CLARK	MANAGER	7839	1981-06-09	2450	NULL	10
7788	SCOTT	ANALYST	7566	1982-12-09	30000	NULL	20
7839	KING	PRESIDENT	NULL	1981-11-17	25000	NULL	10
7844	TURNER	SALESMAN	7698	1981-09-08	45500	0	30
7876	ADAMS	CLERK	7788	1983-01-12	21100	NULL	20
7900	JAMES	CLERK	7698	1981-12-03	25950	NULL	30
7902	FORD	ANALYST	7566	1981-12-03	63000	NULL	20
7934	MILLER	CLERK	7782	1982-01-23	75300	NULL	10

COUNT():

- COUNT function is used to **Count the number of rows** in a database table. It can work on both numeric and non-numeric data types.
- COUNT function uses the COUNT(*) that returns the count of all the rows in a specified table.
- COUNT(*) considers duplicate and Null.
- When we apply COUNT() on a column then NULL values are ignored.

SYNTAX:

```
SELECT COUNT(*) FROM table_name WHERE condition;
```

or

```
SELECT COUNT([ALL DISTINCT] column_name)
```

```
FROM table_name
```

```
WHERE condition;
```

Examples

select count(*) from emp; 14

select count(comm) from emp; 4

select count(job) from emp; 14

select count(distinct job) from emp; 5

select count(ename) from emp where deptno=30; 6

Select count(ename) "No. of Employees", deptno from emp group by deptno;

o/p-	No of Empl	DeptNo
	3	10
	5	20
	6	30

select count(ename) "No. of Employees", deptno from emp group by deptno having count(ename)>3;

o/p-	Empl	DeptNo
	5	20
	6	30

SUM()

- It is used to calculate the total of a numeric column. It adds up all non-NULL values in that column for
Example, **SUM(column_name)** returns sum of all non-NULL values in the specified column.

SYNTAX:

- **SELECT SUM(COL_NAME)** FROM table_name [WHERE
CONDITION] [GROUP BY COL_NAME] [HAVING
CONDITION];

select sum(sal) from emp; 299025.00

select sum(sal) from emp where job='analyst'; 93000.00

select sum(sal) from emp where deptno=10; 102750.00

select job, sum(sal) from emp group by job;

```
mysql> select job, sum(sal) from emp group by job;
```

job	sum(sal)
CLERK	123150.00
SALESMAN	49600.00
MANAGER	8275.00
ANALYST	93000.00
PRESIDENT	25000.00

5 rows in set (0.00 sec)

select deptno, sum(sal) from emp group by deptno having sum(sal)>100000;

```
mysql> select deptno, sum(sal) from emp group by deptno having sum(sal)>100000;
```

deptno	sum(sal)
10	102750.00
20	117875.00

2 rows in set (0.00 sec)

AVG()

- It is used to calculate average value of a numeric column. It divides sum of all non-NULL values by the number of non-NULL rows for
Example, **AVG(column_name)** returns average of all non-NULL values in the specified column.

SYNTAX:

```
SELECT AVG(COL_NAME) FROM table_name  
[WHERE    CONDITION][GROUP BY COL_NAME]  
[HAVING CONDITION];
```

Example:

- `select avg(sal) from emp;` 21358.92
- `select avg(sal) from emp where job='analyst';` 123150.00
- `select deptno, avg(sal) from emp group by deptno having avg(sal)>15000;`

MAX() and MIN()

- The MIN() function returns the smallest value of the selected column.
- The MAX() function returns the largest value of the selected column

SYNTAX:

```
SELECT MIN(COL_NAME) FROM table_name  
[WHERE CONDITION][GROUP BY COL_NAME]  
[HAVING CONDITION];
```

```
SELECT MAX(COL NAME) FROM table_name  
[WHERE CONDITION][GROUP BY COL_NAME]  
[HAVING CONDITION];
```


Example:

select min(sal) from emp;

800.00

select max(sal) from emp where job='analyst';

63000.00

select deptno, max(sal), min(sal) from emp group by deptno;

deptno	max(sal)	min(sal)
10	75300.00	2450.00
20	63000.00	800.00
30	45500.00	1250.00

3 rows in set (0.00 sec)

LOGICAL DATABASE DESIGN

DBMS -UNIT-2

LOGICAL DATABASE DESIGN: ER TO RELATIONAL

- choose a DBMS to implement our database design, and convert the conceptual database design into a database schema in the data model of the chosen DBMS. We will consider only relational DBMS
- We now describe how to translate an ER diagram into a collection of tables with associated constraints, that is, a relational database schema.
- Logical database design is the process of deciding how to arrange the attributes of the entities in a given business environment into database structures, such as the tables of a relational database
-
- The first step before applying the rules in relational data model is converting the conceptual design to a form suitable for relational logical model, which is in a form of tables.

Converting ER Diagram to Relational Tables

- Three basic rules to convert ER into tables or relations:

Rule 1: Entity Names will automatically be table names

Rule 2: Mapping of attributes: attributes will be columns of the respective tables.

Rule 3: Relationships: relationship will be mapped by using a foreign key attribute. Foreign key is a primary or candidate key of one relation used to create association between tables.

Entity Sets to Tables

- An entity set is mapped to a relation in a straight forward way: Each attribute of the entity set becomes an attribute of the table.
- Note that we know both the domain of each attribute and the (primary) key of an entity set.

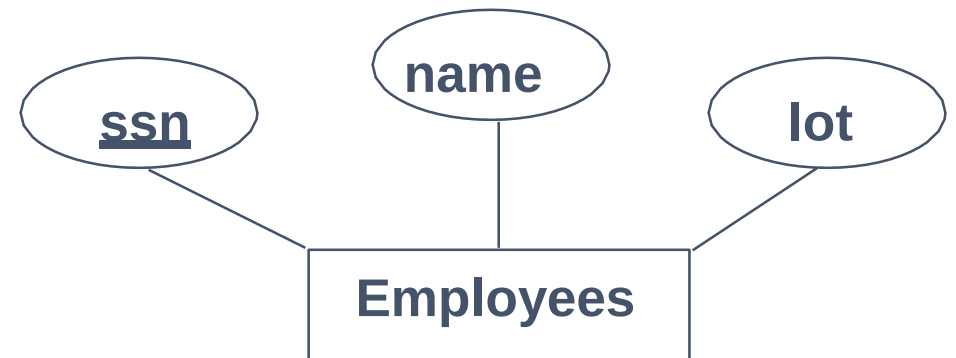
CREATE TABLE Employees

(ssn VARCHAR(11),
name VARCHAR(20),
lot INT,

PRIMARY KEY

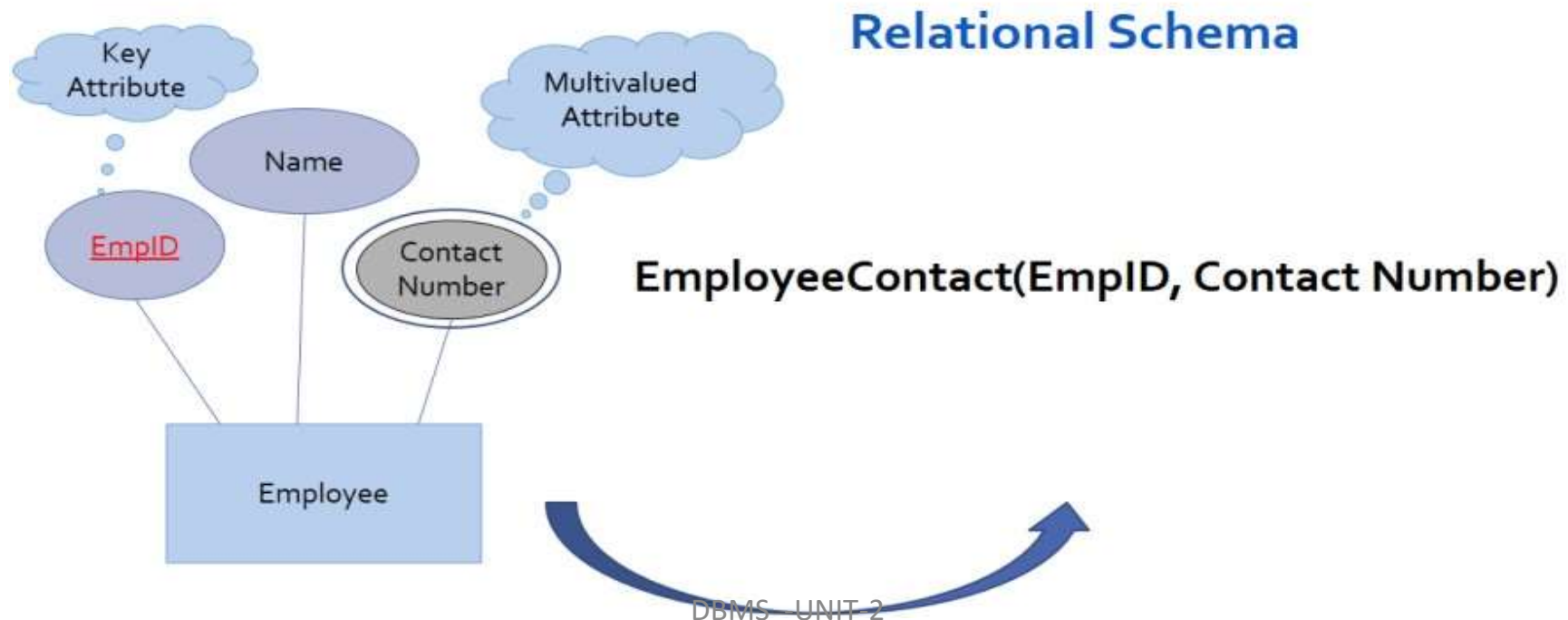
(ssn)

);

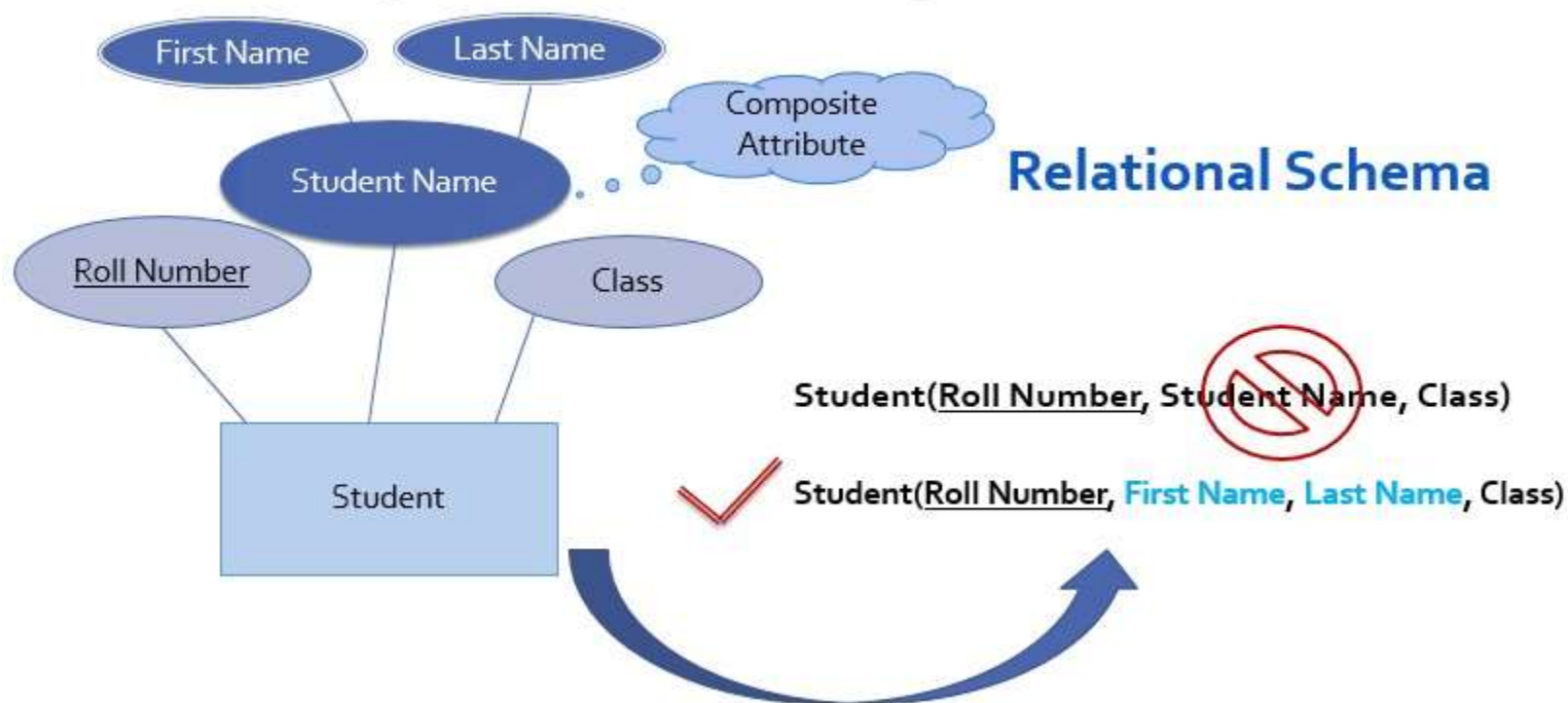


Entity set with multi valued attribute

- Separate relation is created for multivalued attribute
 - separate relation is created for multivalued attribute in which
 - 1.Key attribute and multivalued attribute of entity set becomes primary key of relation.
 2. Separate relation employee is created with remaining attributes.
- Due to this instead of repeating all attributes of entity now only one attribute is need to repeat.



Entity set with Composite attribute:

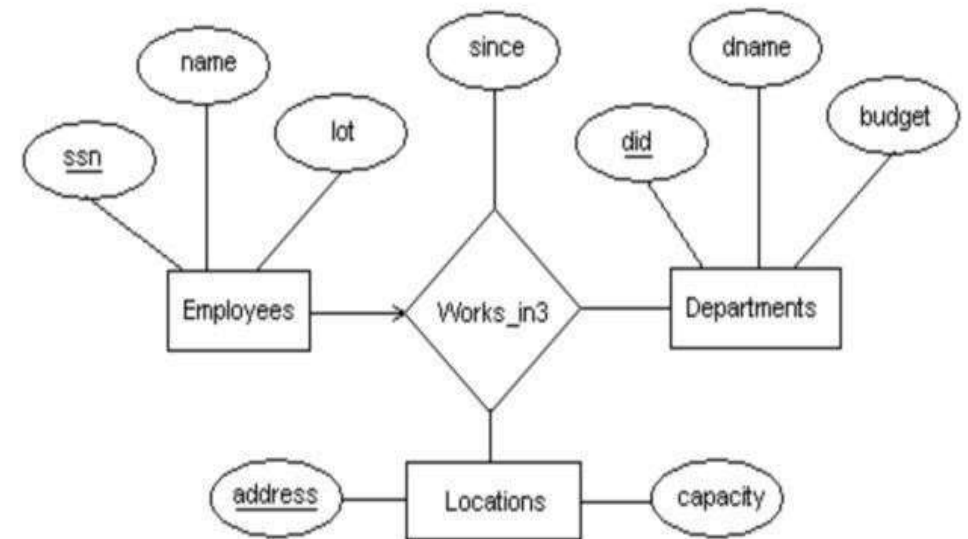


Relationship Sets (without Constraints) to Tables

- A relationship set, like an entity set, is mapped to a relation in the relational model.
- To represent a relationship, we must be able to identify each participating entity and give values to the descriptive attributes of the relationship.
- Thus, the attributes of the relation include:
 - The primary key attributes of each participating entity set, as foreign key fields.
 - The descriptive attributes of the relationship set.

- All the available information about the Works_In3 table is captured by the following SQL definition:

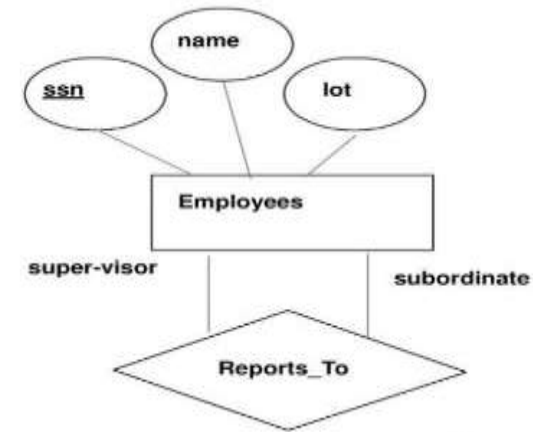
```
CREATE TABLE Works_In3 ( ssn CHAR(11), did INTEGER,
address CHAR(20) ,
since DATE,
PRIMARY KEY (ssn, did, address),
FOREIGN KEY (ssn) REFERENCES Employees (ssn),
FOREIGN KEY (address) REFERENCES Locations(address),
FOREIGN KEY (did) REFERENCES Departments(did));
```



- Finally, consider the Reports_To relationship set.
- The role indicators *supervisor* and *subordinate* are used to create meaningful field names in the CREATE statement for the Reports_To table:

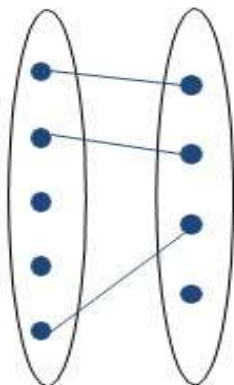
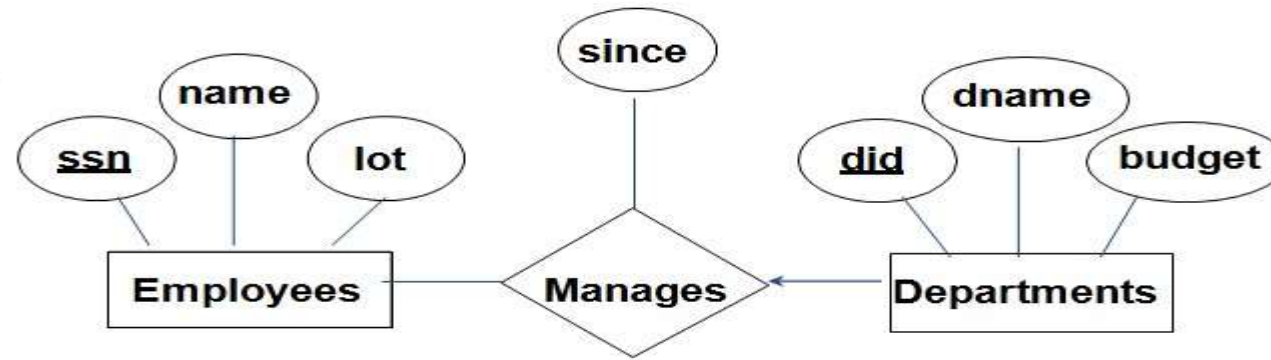
```
CREATE TABLE Reports_To ( supervisor_ssn CHAR (11),
subordinate_ssn CHAR (11) ,
PRIMARY KEY (supervisor_ssn, subordinate_ssn),
FOREIGN KEY (supervisor_ssn) REFERENCES Employees(ssn),
FOREIGN KEY (subordinate_ssn) REFERENCES Employees(ssn) )
```

- Observe that we need to explicitly name the referenced field of Employees because the field name differs from the name(s) of the referring field(s).

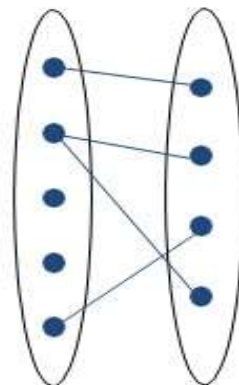


Translating Relationship Sets with Key Constraints

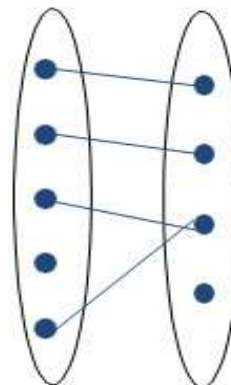
- Review: Key Constraints
- Each dept has at most one manager, according to the key constraint on Manages.



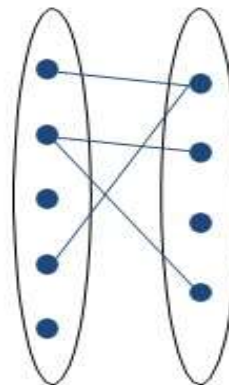
1-to-1



1-to Many



Many-to-1



Many-to-Many

Translation to relational model?

- Because each department has at most one manager, no two tuples can have the same *did* value but differ on the *ssn* value.
- A consequence of this observation is that *did* is itself a key for Manages; indeed, the set *did*, *ssn* is not a key (because it is not minimal).
- The Manages relation can be defined using the following SQL statement:

```
CREATE TABLE Manages(  
  ssn CHAR(11), did INTEGER,  
  since DATE, PRIMARY KEY (did),  
  FOREIGN KEY (ssn) REFERENCES Employees (ssn),  
  FOREIGN KEY (did) REFERENCES  
  Departments(did))
```

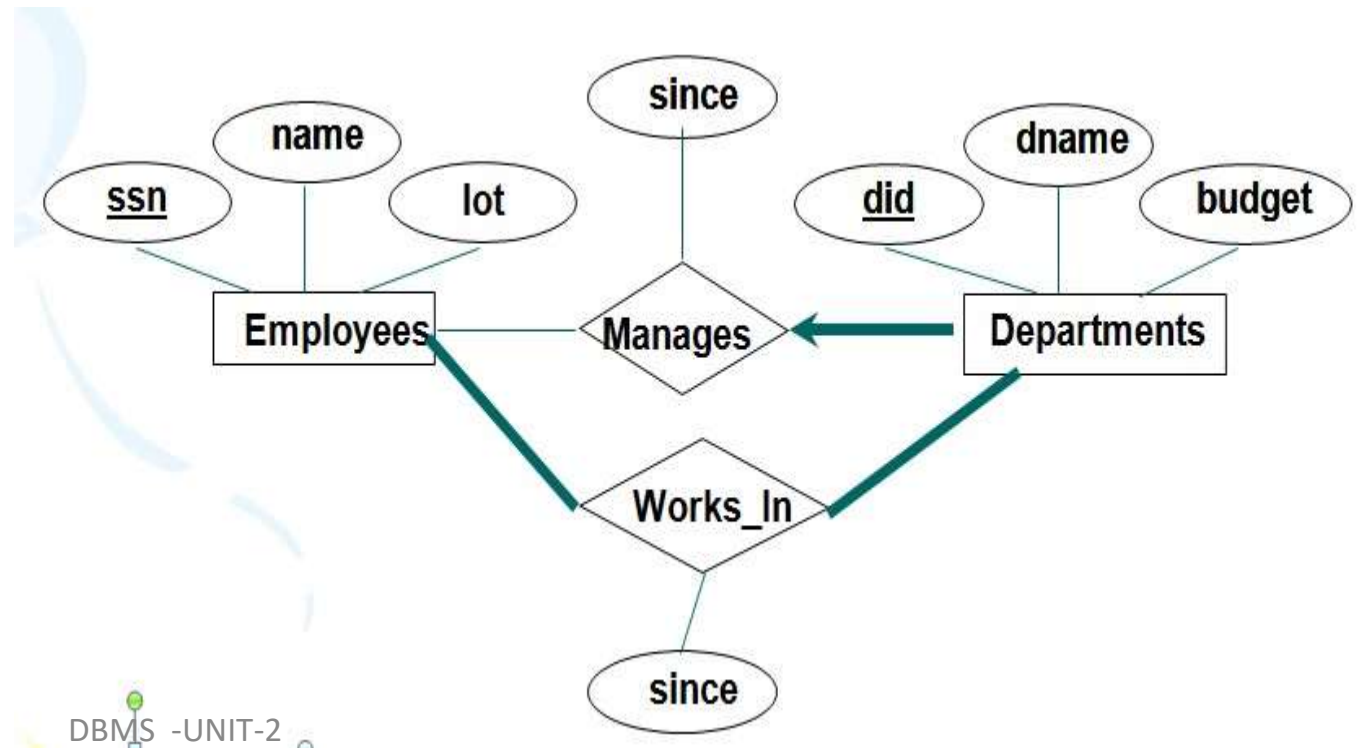
- **A second approach:** Since each department has a unique manager, we could instead combine Manages and Departments.
- This approach eliminates the need for a separate Manages relation, and queries asking for a department's manager can be answered without combining information from two relations.
- The only drawback to this approach is that space could be
- wasted if several departments have no managers.
- In this case the added fields would have to be filled with
- *null* values.

```
CREATE TABLE Dept_Mgr( did INTEGER,  
  dname CHAR(20), budget REAL,  
  ssn CHAR(11),  
  since DATE,  
  PRIMARY KEY (did),  
  FOREIGN KEY (ssn) REFERENCES  
    Employees(ssn))
```

Translating Relationship Sets with Participation Constraints

- Every department is required to have a manager, due to the participation constraint, and at most one manager, due to the key constraint.

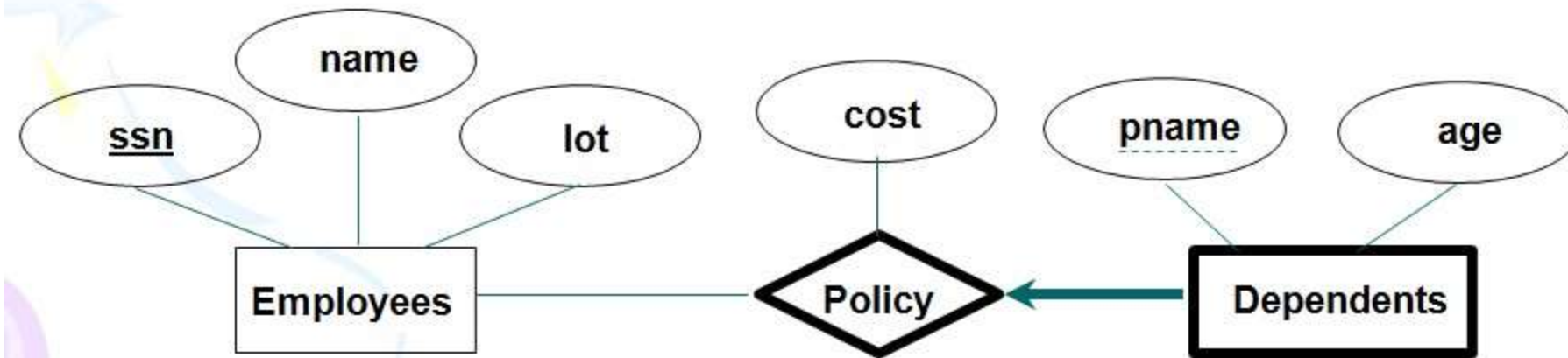
```
CREATE TABLE Dept_Mgr(  
  did    INTEGER,  
  dname  CHAR(20),  
  budget REAL,  
  ssn    CHAR(11) NOT NULL,  
  since  DATE,  
  PRIMARY KEY (did),  
  FOREIGN KEY (ssn)  
  REFERENCES Employees(ssn),  
  ON DELETE NO ACTION)
```



- It also captures the participation constraint that every department must have a manager:
- Because *ssn* cannot take on *null* values, each tuple of Dept_Mgr identifies a tuple in Employees (who is the manager).
- The **NO ACTION** specification, which is the default and need not be explicitly specified, ensures that an **Employees tuple cannot be deleted** while it is pointed to by a Dept_Mgr tuple.
- If we wish to delete such an Employees tuple, we must first change the Dept_Mgr tuple to have a new employee as manager.

Translating Weak Entity Sets

- A weak entity set always participates in a one-to-many binary relationship and has a key constraint and total participation.
- A Dependents entity can be identified uniquely only if we take the key of the *owning* Employees entity and the *pname* of the Dependents entity, and the Dependents entity must be deleted if the owning Employees entity is deleted.

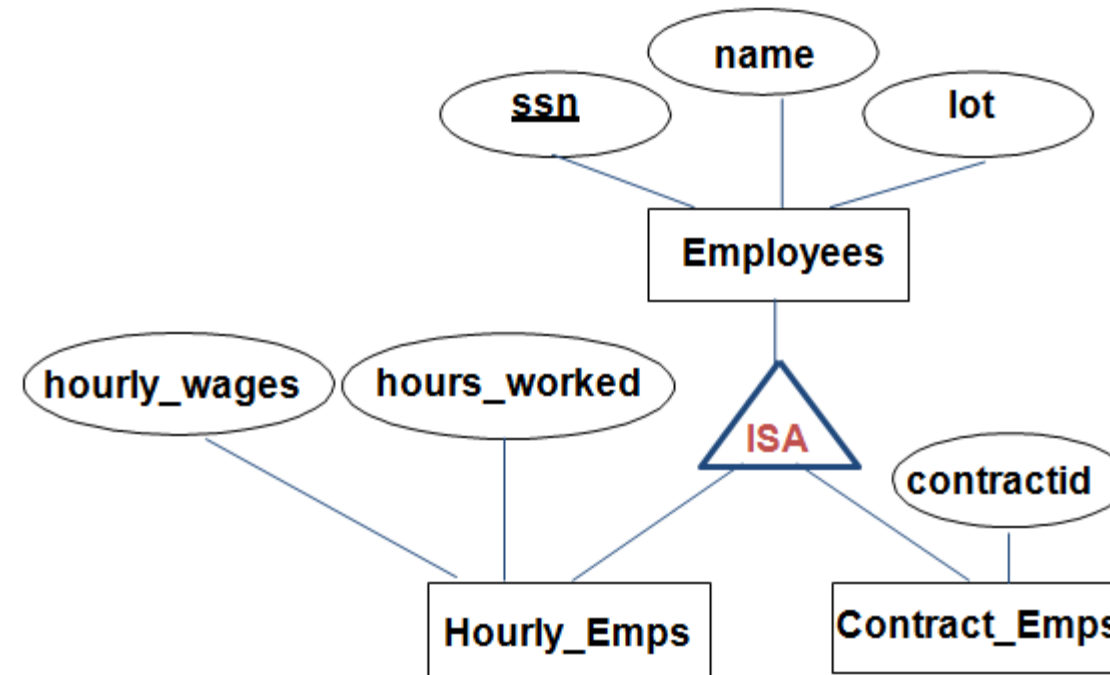


- Weak entity set and identifying relationship set are translated into a single table.
- When the owner entity is deleted, all owned weak entities must also be deleted.

```
CREATE TABLE Dep_Policy ( pname CHAR(20),  
age      INTEGER,  
cost     REAL,  
ssn      CHAR(11) NOT NULL,  
PRIMARY KEY (pname, ssn),  
FOREIGN KEY (ssn) REFERENCES Employees(ssn),  
ON DELETE CASCADE)
```

Translating Class Hierarchies

- *Overlap constraints:*
- Can Joe be an Hourly_Emps as well as a Contract_Emps entity?
(*Allowed/disallowed*)
- *Covering constraints:*
- Does every Employees entity also have to be an Hourly_Emps or a Contract_Emps entity? (*Yes/no*)



- **General approach:**

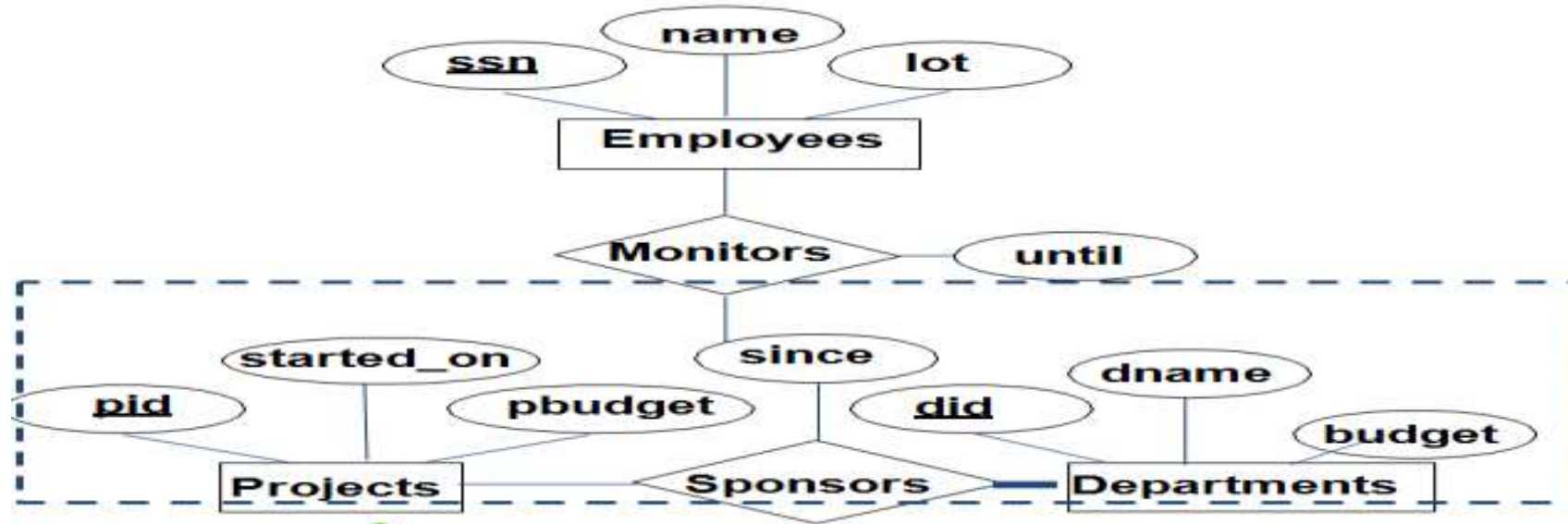
- 3 relations: Employees, Hourly_Emps and Contract_Emps.
 - *Hourly_Emps*: Every employee is recorded in Employees.
For hourly emps, extra info recorded in Hourly_Emps (*hourly_wages*, *hours_worked*, *ssn*); must delete Hourly_Emps tuple if referenced Employees tuple is deleted).
 - Queries involving all employees is easy, those involving just Hourly_Emps require a join to get some attributes.

- **Alternative:** Just Hourly_Emps and Contract_Emps.

- *Hourly_Emps*: *ssn*, *name*, *lot*, *hourly_wages*, *hours_worked*.
- Each employee must be in one of these two subclasses.

Translating ER Diagrams with Aggregation

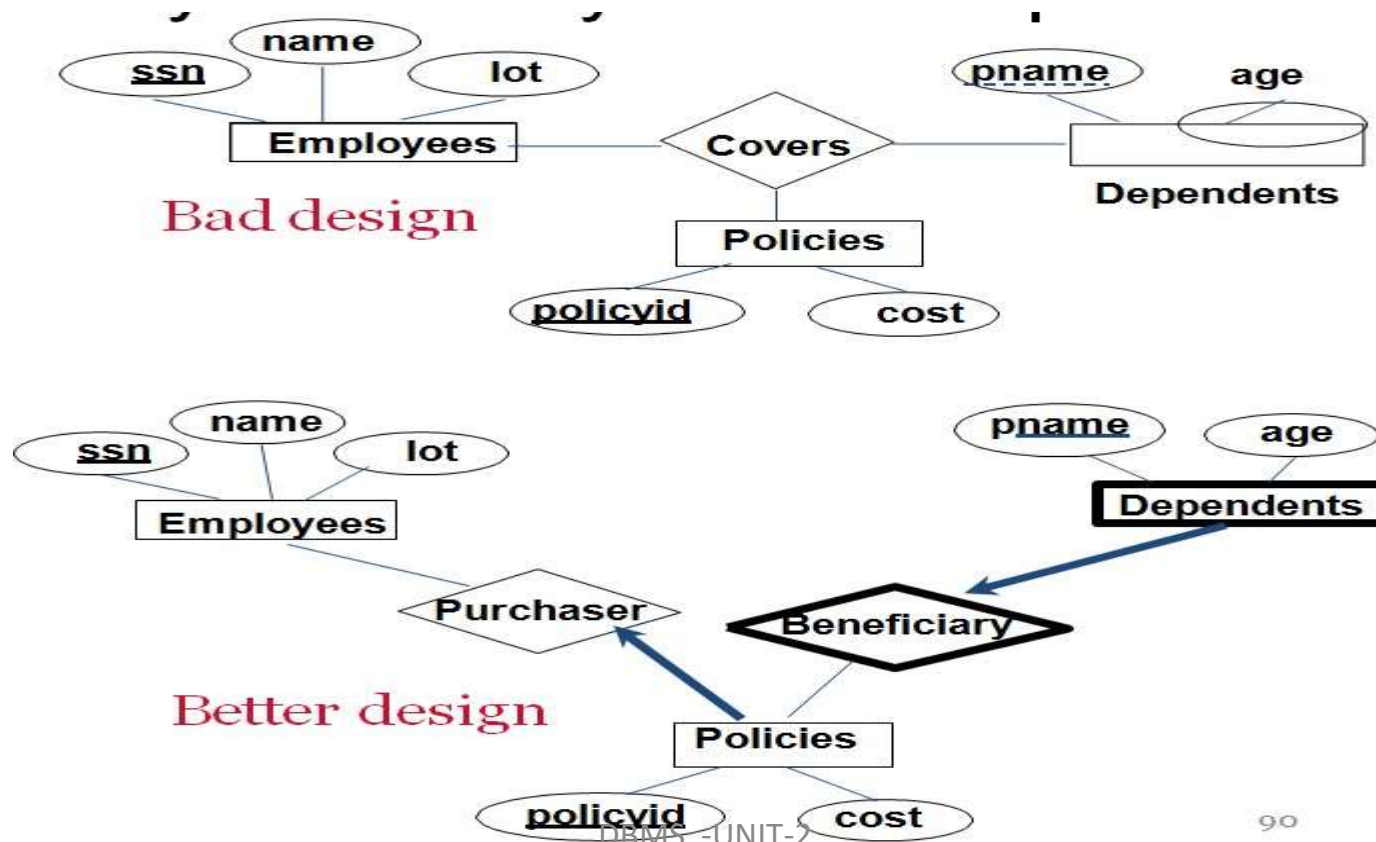
- For the Monitors relationship set, we create a relation with the following attributes: the key attributes of Employees (*ssn*), the key attributes of Sponsors (*did*, *pid*), and the descriptive attributes of Monitors (*until*).



- There is a special case in which this translation can be refined by dropping the Sponsors relation.
- Consider the Sponsors relation. It has attributes *pid*, *did*, and *since*; and in general we need it (in addition to Monitors) for two reasons:
 1. We have to record the descriptive attributes (in our example, *since*) of the Sponsors relationship.
 2. Not every sponsorship has a monitor, and thus some (*pid*, *did*) pairs in the Sponsors relation may not appear in the Monitors relation.
- However, if Sponsors has no descriptive attributes and has total participation in Monitors, every possible instance of the Sponsors relation can be obtained from the (*pid*, *did*) columns of Monitors; Sponsors can be dropped.

ER to Relational: Additional Examples

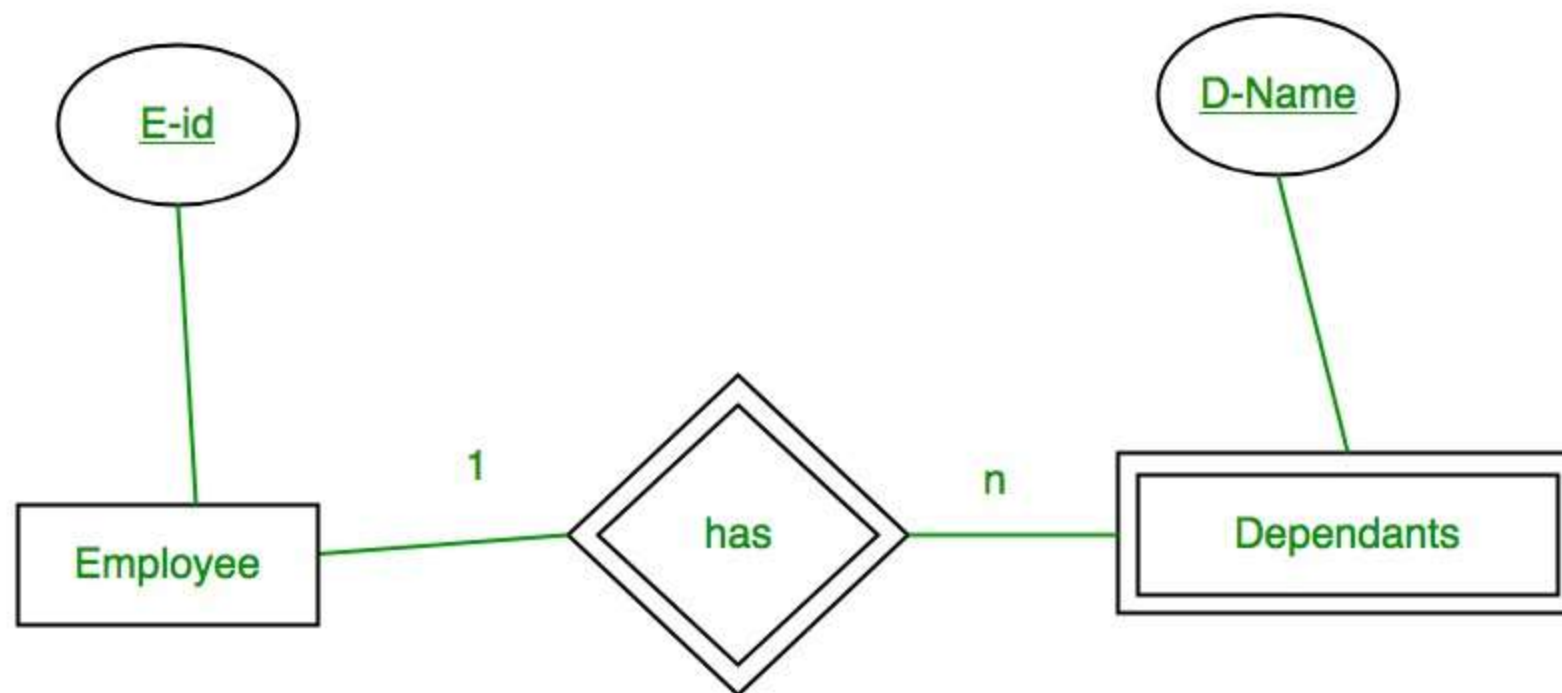
- The key constraints allow us to combine Purchaser with Policies and Beneficiary with Dependents.
- Participation constraints lead to **NOT NULL** constraints.
- What if Dependents is a weak entity set?



```
CREATE TABLE Policies ( policyid INTEGER, cost REAL,  
ssn CHAR(11) NOT NULL,  
PRIMARY KEY (policyid).  
FOREIGN KEY (ssn) REFERENCES Employees(ssn),  
ON DELETE CASCADE)
```

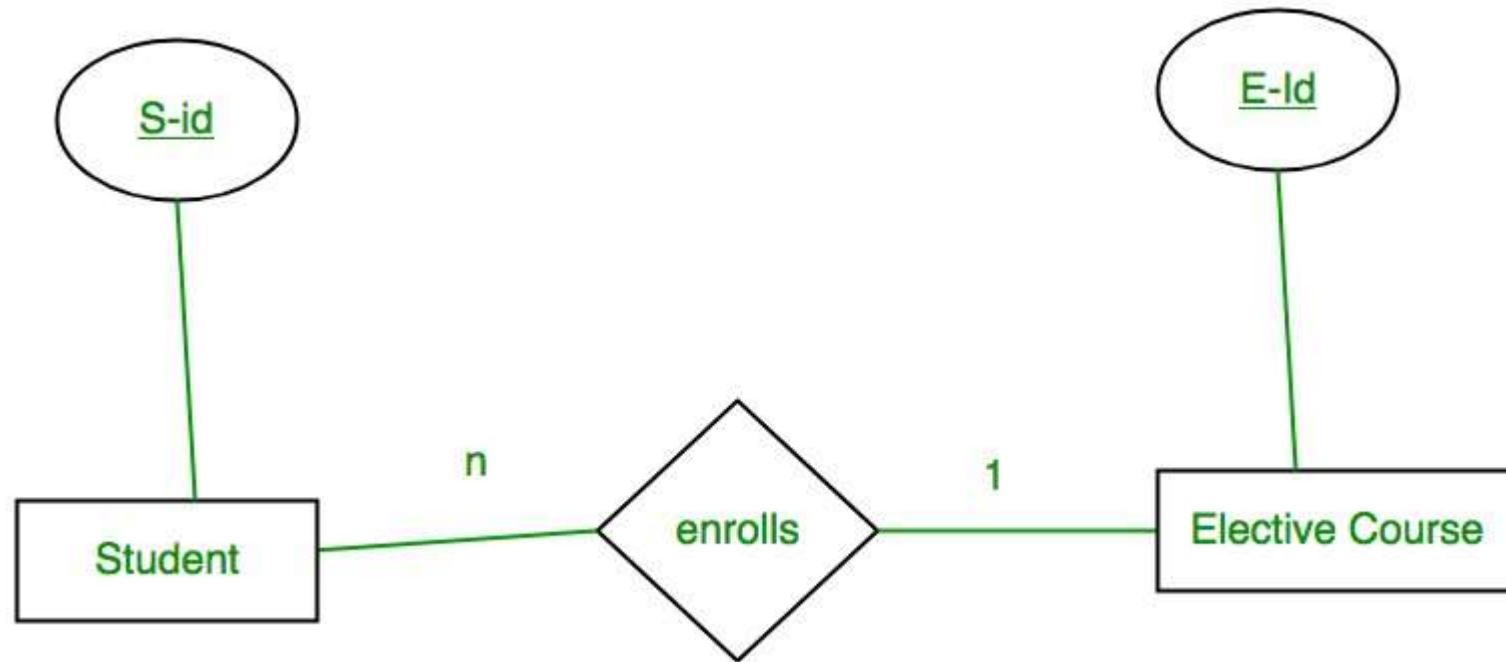
```
CREATE TABLE Dependents ( pname CHAR(20), age INTEGER,  
policyid INTEGER, PRIMARY KEY (pname, policyid).  
FOREIGN KEY (policyid) REFERENCES Policies(policyid),  
ON DELETE CASCADE)
```

1:M (one to many) Relationship:



- In this scenario, an employee can have many dependents and one dependent can depend on one employee.
- A dependent does not have any existence without an employee (e.g; you as a child can be dependent of your father in his company).
- So it will be a weak entity and its participation will always be total.
- **Weak Entity does not have key of its own.**
- So its key will be combination of key of its identifying entity (E-Id of Employee in this case) and its partial key (D-Name).
- First Convert each entity and relationship to tables.
- Employee table corresponds to Employee **Entity with key as E-Id.**
- Similarly Dependents table corresponds to Dependent Entity with key as D-Name and E-Id.
- Has Table represents relationship between Employee and Dependents (Which employee has which dependents).
- So it will take attribute E-Id from Employee and D-Name from **Dependents.**

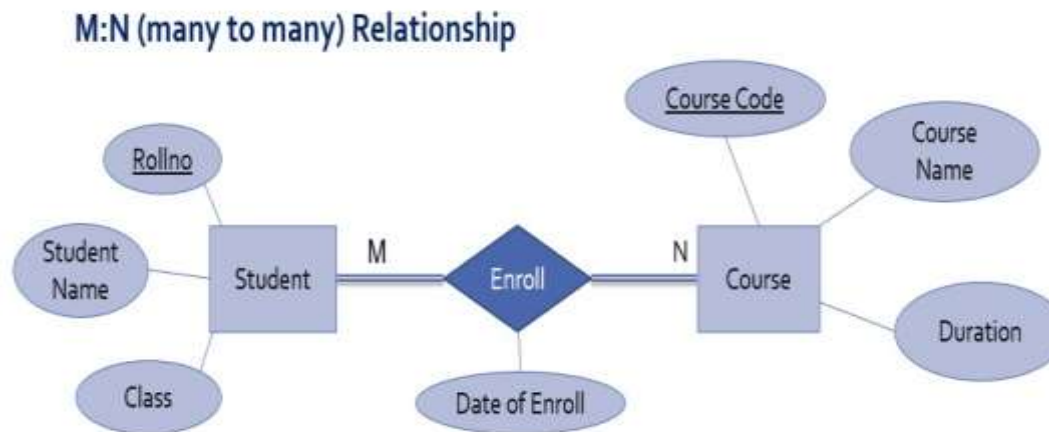
M:1 (many to one) Relationship:



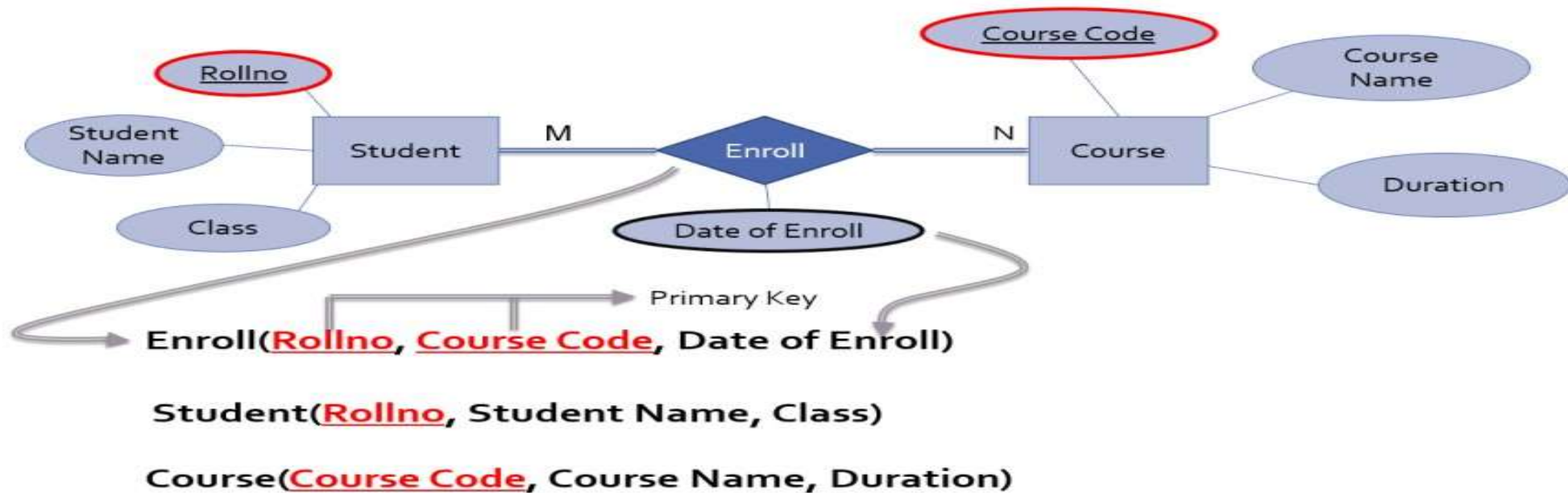
- In this scenario, every student can enroll only in one elective course but for an elective course there can be more than one student.
- First Convert each entity and relationship to tables.
- Student table corresponds to Student Entity with key as S-Id.
- Similarly Elective_Course table corresponds to Elective_Course Entity with key as E-Id.
- Enrolls Table represents relationship between Student and Elective_Course (Which student enrolls in which course).
- So it will take attribute S-Id from Student and E-Id from Elective_Course.

M:N (many to many) Relationship:

- Consider same relationship set enrolled exist between entity sets student and course ,which means multiple student can enroll in multiple courses.

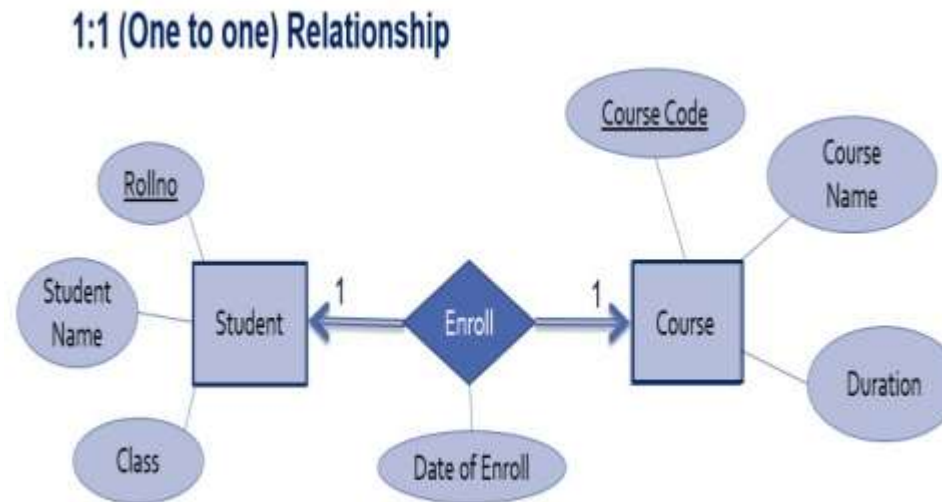


- To convert this Relationship set into relational schema,
 - Relationship set is mapped as separate relation
 - Key attributes of participating entity sets are mapped as primary key for that relation
 - Attribute of relationship set becomes simple attributes for that relation
 - And separate relation is created for other participating entities

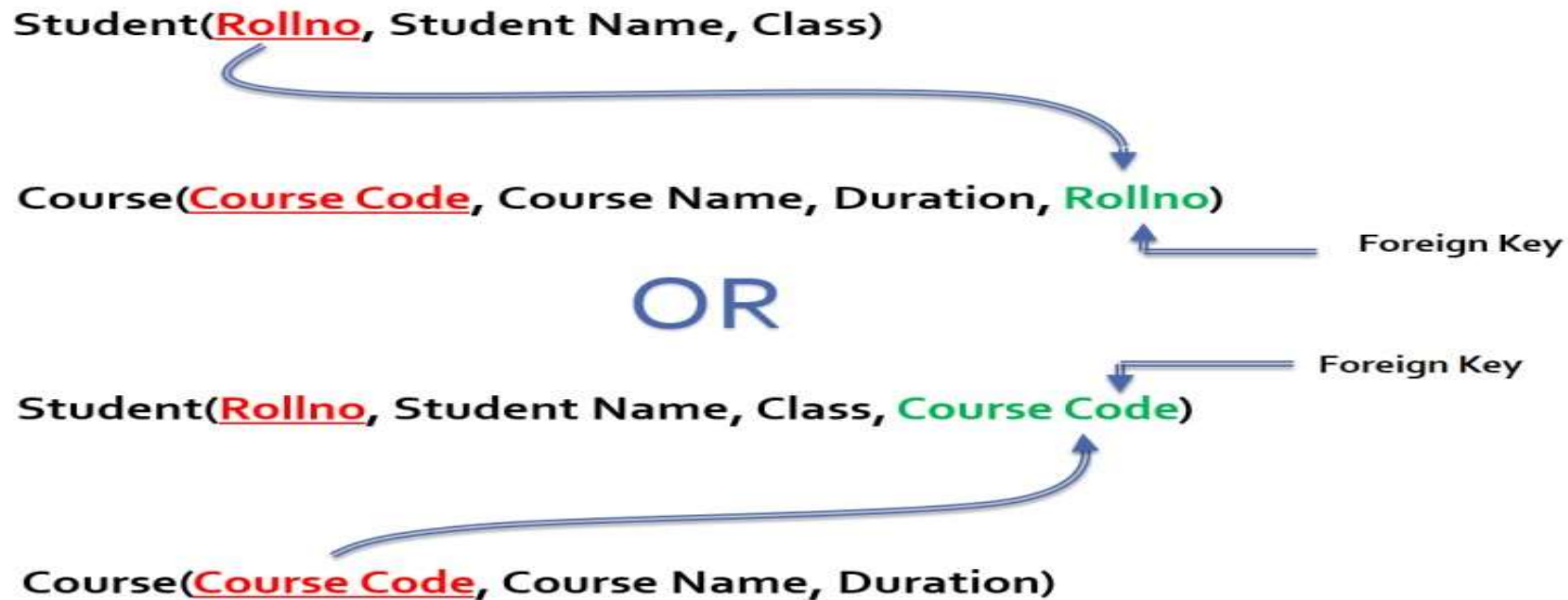


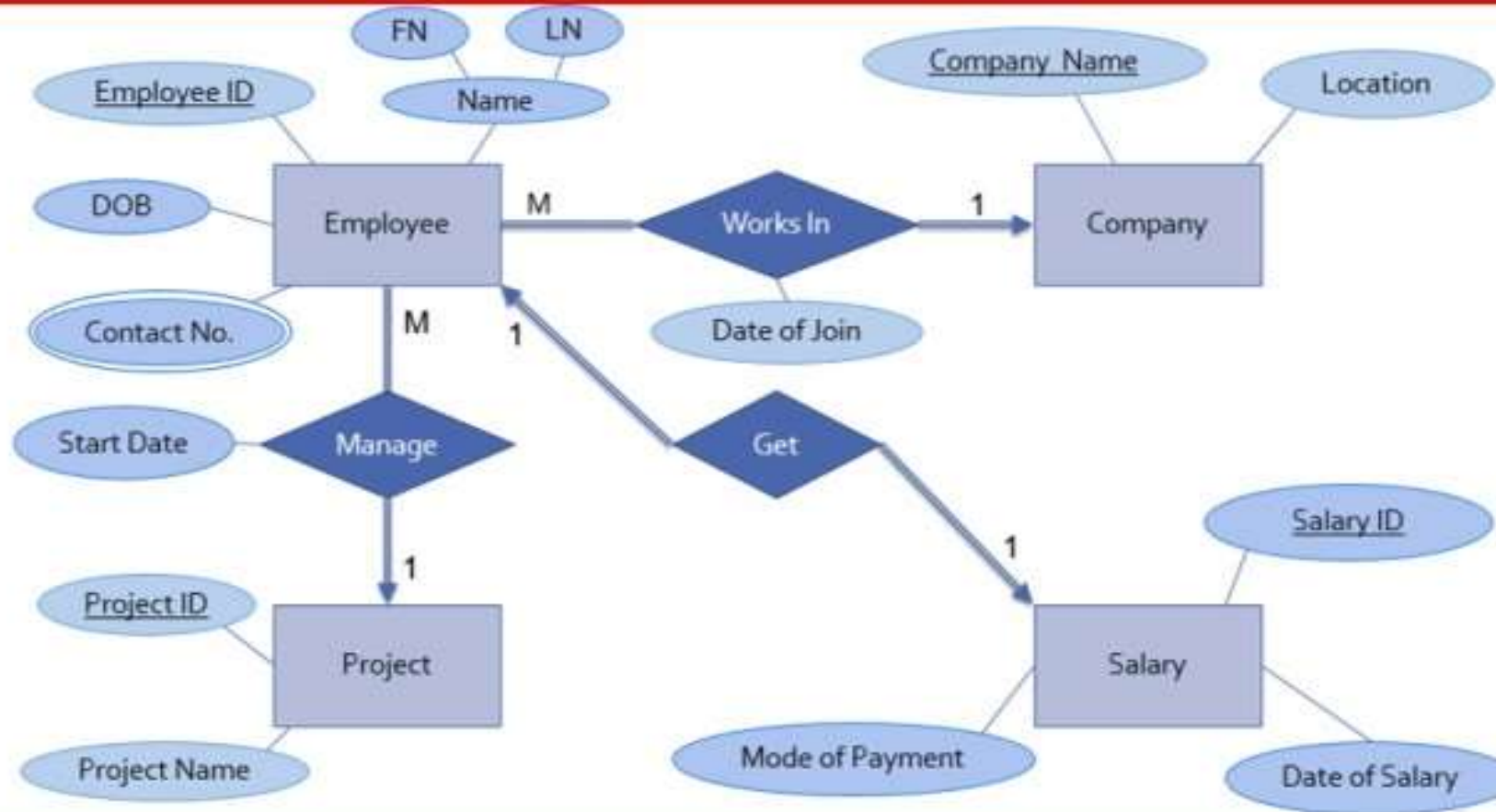
1:1 (one to one) Relationship:

- Consider same relationship set enroll exist between entity sets student and course ,which means one student can enroll in only one courses



- To convert this Relationship set into relational schema,
 - Separate relation is created for all participating entity sets.
 - Primary Key of Relation Student can be act as foreign key for relation Course
OR Primary Key of Relation Course act as foreign key for relation Student.





ER Diagram

Employee(Employee ID , DOB , FN , LN , Salary ID)
 EmployeeContact(Employee ID , Contact No)
 Company(Company Name , Location , Employee ID , Date of Join)
 Project(Project ID , Project Name , Employee ID , Start Date)
 Salary(Salary ID , Mode of Payment , Date of Salary , Employee ID)

Relational Model

INTRODUCTION TO VIEWS

DBMS -UNIT-2

View Definition

- A relation that is not of the conceptual model but is made visible to a user as a “virtual relation” is called a **view**.
- Views in SQL are considered as a **virtual table**. A view also contains rows and columns.
- To create the view, we can select the fields from one or more tables present in the database.
- A view can either have specific rows based on certain condition or all the rows of a table.
- Views can be used to present necessary information (or a summary), while hiding details in underlying relation(s).
- When data redundancy is to be kept minimum while maintaining security

Advantages of Views

- Simplicity – Users don't need to remember complex queries.
- Security – Hide sensitive columns (like salary, password).
- Consistency – Same query logic reused by many users.
- Data abstraction – Users interact with views, not raw tables..

Types of Views

1. Simple View:

- Based on a single table.
- Supports DML operations.

2. Complex View

- Based on multiple tables (joins, subqueries).
- Generally, not updatable.

Creating Simple VIEW

- A view can be created using the CREATE VIEW statement.
- We can create a view from a single table.

Syntax:

```
CREATE VIEW view_name AS  
SELECT column1, column2.....  
FROM table_name  
WHERE condition;
```

Note: view_name: Name for the View

table_name: Name of the table

condition: Condition to select rows

Creating View from a single table

- In this example, we create a View named Details View from the table Student_Detail.
- Query:

```
CREATE VIEW DetailsView AS  
SELECT NAME, ADDRESS  
FROM StudentDetails  
WHERE S_ID < 4;
```

Just like table query, we can query the view to view the data.

```
SELECT * FROM DetailsView;
```

Creating Complex View from multiple tables

- View from multiple tables can be created by simply include multiple tables in the SELECT statement.
- SYNTAX:

```
CREATE VIEW MarksView AS
```

```
SELECT Table1.colname1, Table1.colname2, Table2.colname3
```

```
FROM Table1,Table2
```

```
WHERE table1.colname4=table2.colname2;
```

Example

In the given example, a view is created named MarksView from two tables Student_Detail and Student_Marks.

```
CREATE VIEW MarksView AS  
SELECT StudentDetails.NAME, StudentDetails.ADDRESS, StudentMarks.MARKS  
FROM StudentDetails, StudentMarks  
WHERE StudentDetails.NAME = StudentMarks.NAME;
```


Querying a View

- You can treat a view like a table
- To display data of View MarksView:

```
SELECT * FROM MarksView;
```

Views and Security

Given YoungStudents, but not Students or Enrolled, we can find students *s* who have are enrolled, but not the *cid*'s of the courses they are enrolled in.

```
CREATE VIEW YoungActiveStudents AS SELECT S.name as name, E.grade as grade  
FROM Students as S, Enrolled as E  
WHERE S.sid = E.sid and S.age<21
```

- A view is just a relation, but we store a *definition*, rather than a set of tuples.

Eg: Create view Students_B as select S.sid,S.name,E.cid From Students as S, Enrolled as E where S.sid=E.sid

Update SQL View

- The SQL view created can also be modified. We can do the following operations with the SQL VIEW.
- But, all views are not updatable. A SQL view can be updated if the following conditions are satisfied.
 - 1.The view is defined based on only one table.
 - 2.The view should not have any field which is made of an aggregate function.
 - 3.The view must not have GROUP BY, HAVING or DISTINCT clause in its definition.
 - 4.The view should not be created using any nested query.
 - 5.The selected output fields of the view must not use constants, string or value expressions.
 - 6.If you want to update a view based on another view then that view should be updatable

Update SQL View

VIEW Creation:

```
CREATE VIEW high_salary_emps AS  
SELECT empno, ename, sal  
FROM emp  
WHERE sal > 50000;
```

Updating a View:

```
UPDATE high_salary_emps  
SET sal = 90000  
WHERE empno = 101;
```

This updates the underlying table (emp).

Note: If a view is based on a single table without aggregate functions, DISTINCT, GROUP BY, UNION, etc., you can perform INSERT, UPDATE, DELETE.

VIEW Creation **WITH CHECK OPTION**

- Ensures modifications through the view do not violate its condition.

VIEW Creation:

```
CREATE VIEW sales_staff AS  
SELECT empno, ename, job, sal  
FROM emp  
WHERE job = 'SALESMAN'  
WITH CHECK OPTION;
```

Note: *If you try to INSERT or UPDATE an employee with job $\neq$ 'SALESMAN' using this view, MySQL will reject it.*

Altering View

- Directly it is not possible to modify structure of a VIEW.
- But can use the CREATE OR REPLACE VIEW statement to modify the SQL view.

Syntax:

```
CREATE OR REPLACE VIEW view_name AS  
SELECT column1,column2,..  
FROM table_name  
WHERE condition;
```

Example: If we want to update the view marksvew1 and remove the attribute "Age" from in the view then the query would be:

Query

```
mysql> create or replace view marksvew1 as select student detail.name, student marks.marks  
from student_detail, student_marks where student_detail.std_id=student_marks.std_id;
```

Destroying View

- A view can be deleted using the Drop View statement.

Syntax

DROP VIEW view_name;

View_name: Name of the View which we want to delete.

- **Example:**

If we want to delete the View MarksView, we can do this as:

EX: **DROP VIEW MarksView;**

Destroying Multiple Views

Syntax:

```
DROP VIEW view_name1, view_name2, view_name3;
```

Example:

```
DROP VIEW MarksView,SubjectsView;
```

Checking View Definition

Syntax:

```
SHOW CREATE VIEW view_name;
```

Example:

```
SHOW CREATE VIEW high_salary_emps;
```

Thank you!

